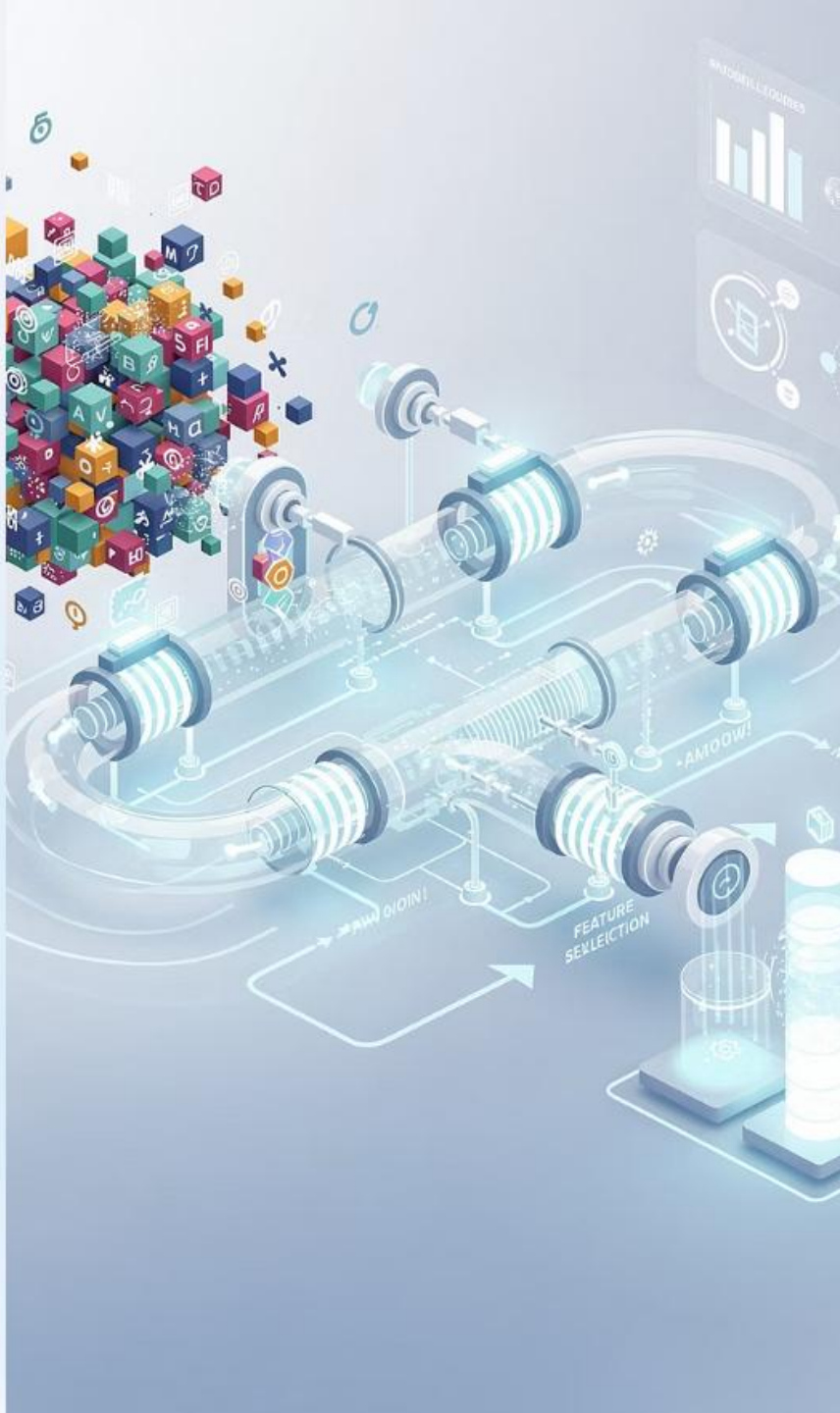
Erasmus Mundus Joint Master Artificial Intelligence for Sustainable Societies (AISS)

Topics on Machine Learning and its Applications

Feature engineering

2nd part

Pedro Strecht



Feature engineering (2nd part)

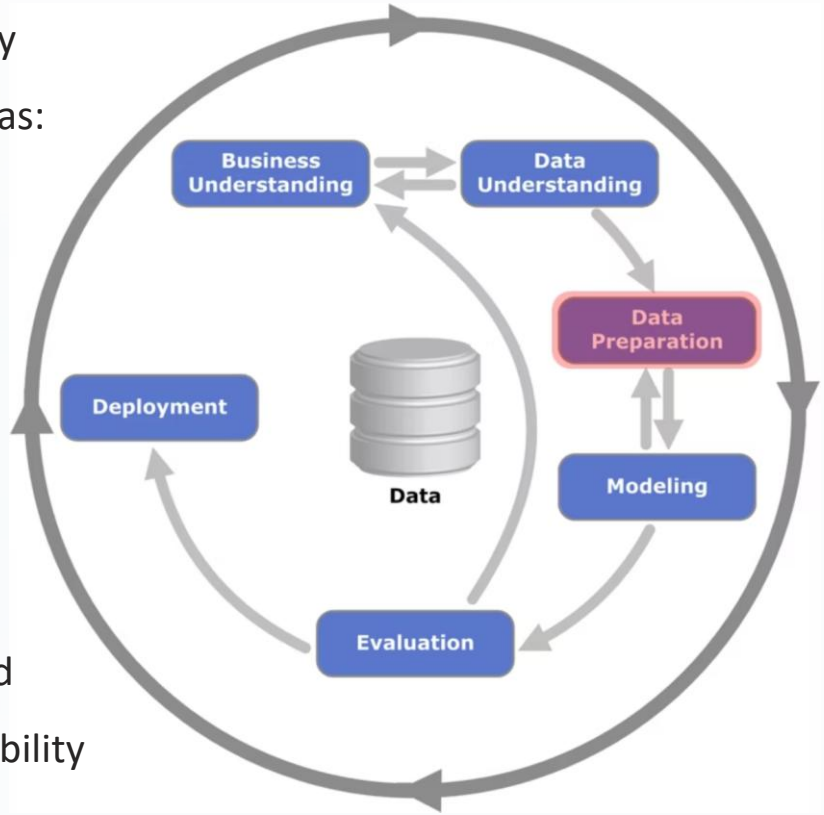
- Dimensionality reduction
- Handling class imbalance

Feature engineering

Introduction

In CRISP-DM **Feature engineering** corresponds to **Data preparation**

- The **data preparation** phase transforms raw data into a refined dataset ready for analysis. This involves a series of steps tailored to the specific data, such as:
  1. **Consolidating** data from various sources;
  2. **Standardizing** column names for consistency;
  3. **Cleaning** data by addressing errors and inconsistencies;
  4. **Identifying and handling** outliers;
  5. **Transforming** data into a suitable format for analysis tools;
  6. **Partitioning** data into training, testing, and validation sets.
- Once finalized, these cleaning and transformation steps are often automated within an **ETL** (Extract, Transform, Load) process for efficiency and reproducibility



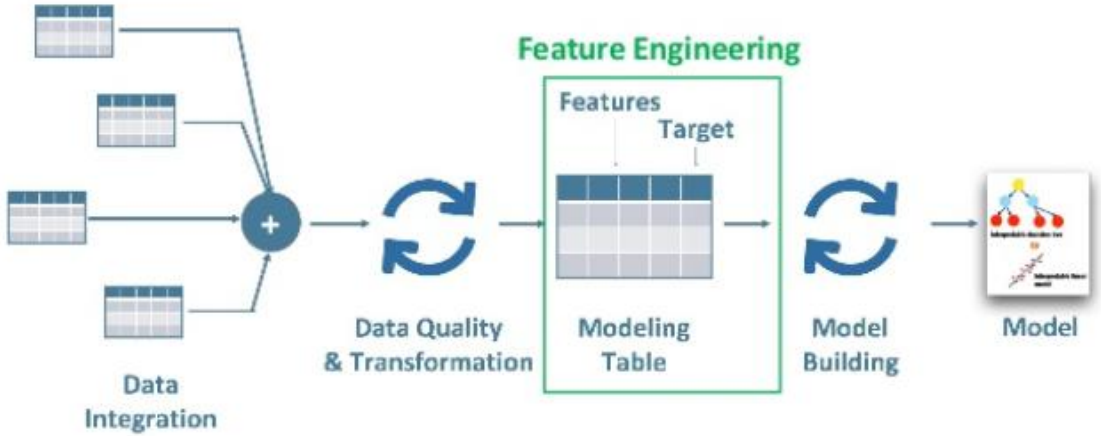
Feature engineering

Dimensionality reduction

- **Feature Engineering** is the critical process of creating, transforming, and selecting variables to improve model performance

- Typically includes tasks that can be grouped a:
  - **Feature creation**
  - **Feature transformation**
  - **Feature selection**
  - **Feature extraction**

Dimensionality reduction



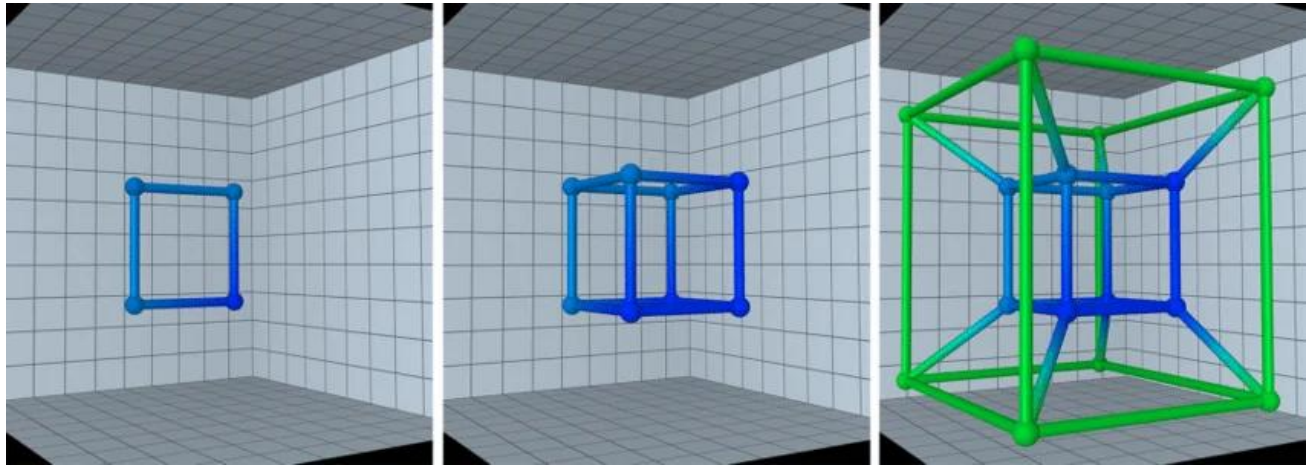
Getting Started with Feature Engineering:
https://www.analyticsvidhya.com/blog/2020/10/getting-started-with-feature-engineering/

Feature engineering

Dimensionality reduction

- **Dimensionality reduction** is the process of reducing the number of features in a dataset while attempting to retain the most relevant information

- As the number of features increases, the **volume of the decision space** grows exponentially, causing data points to become increasingly sparse



The Bridges Baltimore 2015 exhibition of mathematical art, DOI:10.1080/17513472.2016.1178689

Feature engineering

Dimensionality reduction

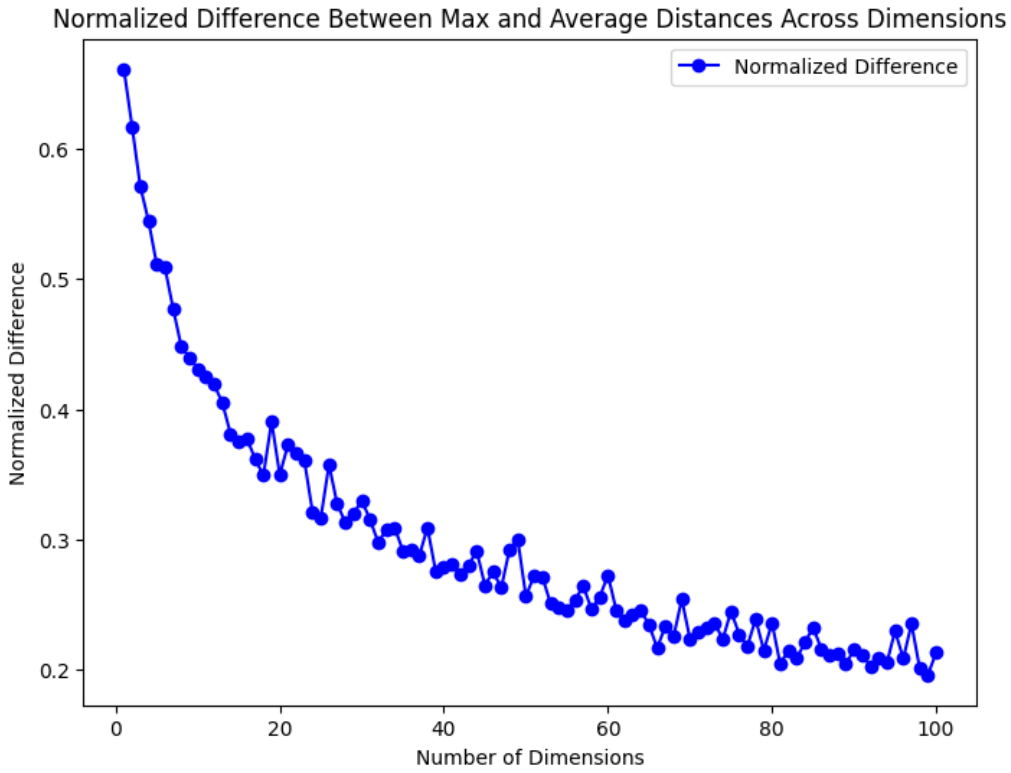
- In high-dimensional spaces, **distance measures become less informative**, which makes it harder for many algorithms to identify meaningful patterns

- This phenomenon is known as the

  "Curse of Dimensionality"

  where high dimensionality can lead to:

  - increased risk of overfitting
  - higher computational cost
  - reduced model generalization



Curse of Dimensionality: An Intuitive Exploration:
https://towardsdatascience.com/curse-of-dimensionality-an-intuitive-exploration-1fbf155e1411/

Feature engineering

Dimensionality reduction

Gr.	Dataset	n	d	μ	d_c	n_{mv}	r_{mv}	t
#1	China's population⋆	73	4	292	0	0	0.00	0.04
	Iris†	150	4	600	0	0	0.00	0.05
	Old faithful⋆	242	2	484	0	0	0.00	0.06
	Phoneme◇	5 404	5	27 020	0	0	0.00	0.16
	2D elastod. metamat.†	20 520	3	61 560	0	0	0.00	0.37
	Stock market@Kraken⋆	32 946 819	2	65 893 638	0	0	0.00	264.89
#2	Wine quality⋆	1 143	12	13 716	0	0	0.00	0.28
	Dry bean†	13 611	16	217 776	0	0	0.00	1.16
	SGEMM GPU kernel perf.†	241 600	18	4 434 800	0	0	0.00	45.85
	Students' dropout†	4 424	37	163 688	0	0	0.00	1.62
	Rice MSC⋆	75 000	106	7 950 000	0	8	0.00	214.05
	Biological response†	3 751	1776	6 661 776	0	0	0.00	78.81
#3	Height of male/Female⋆	199	4	796	1	0	0.00	0.14
	Students perf. in exams⋆	1 000	8	8 000	5	0	0.00	0.20
	Car evaluation†	1 728	7	12 096	7	0	0.00	0.42
	Auction verification†	2 043	9	18 387	1	0	0.00	0.25
	Diabetes†	29 278	2	58 556	1	0	0.00	0.19
	Airlines train◇	10 000 000	10	100 000 000	3	0	0.00	1162.48
#4	Breast cancer Wisconsin†	699	8	5 592	0	16	0.02	0.14
	HCV◇	615	13	7 995	2	26	0.04	0.79
	Astronauts⋆	952	11	10 472	10	952	1.00	13.77
	Mushroom†	8 124	23	186 852	23	2 480	0.31	4.71
	Job change of data scient.⋆	19 158	12	229 896	10	10 203	0.53	262.36
	Adult†	32 561	13	423 293	8	2 399	0.07	31.23

⋆ Kaggle (Kaggle, 2024); † UCI (Markelle Kelly et al., 2023); ◇ OpenML (Vanschoren et al., 2014)

Density Estimation in High-Dimensional Spaces: A Multivariate Histogram Approach
Pedro Strecht, João-Mendes Moreira, Carlos Soares, ADMA 2022, Springer, Cham

https://link.springer.com/chapter/10.1007/978-3-031-22137-_20

- ***"High-dimensional data"*** generally refer to data in which the number of variables is larger than the sample size

- Analyzing such datasets is challenging for classical statistical learning methods because these methods assume scenarios where the sample size grows much larger than the number of variables, an assumption that does not hold in high-dimensional settings

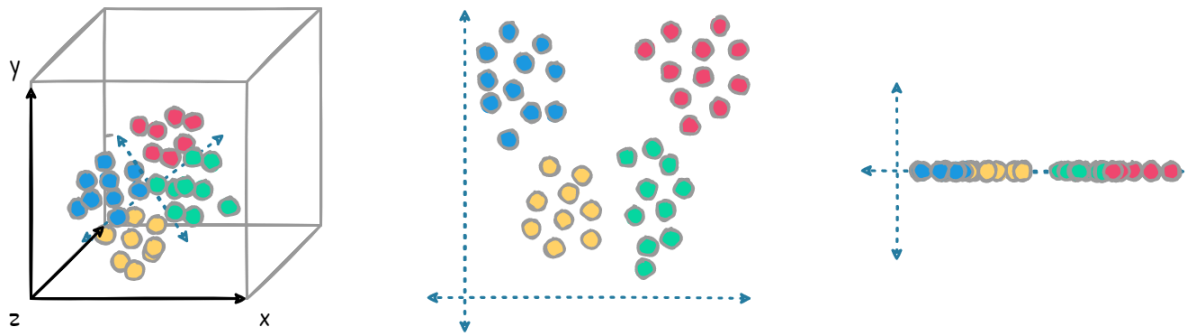
High-Dimensional Statistical Learning: Roots, Justifications, and Potential Machineries, Amin Zollanvari, Cancer Inform, 2016

https://pubmed.ncbi.nlm.nih.gov/27081307/

Feature engineering

Dimensionality reduction

- **Reducing dimensionality** is a fundamental step when working with high-dimensional datasets
- **Not all features contribute equally**, making it essential to distinguish between relevant and less relevant variables
- An inadequate reduction may lead to the **loss of important predictive information**
- The main difficulty lies in decreasing the number of dimensions **while preserving the variables** that best support the predictive task



What is Dimensionality Reduction? A Guide.: https://blog.roboflow.com/what-is-dimensionality-reduction/

Feature engineering

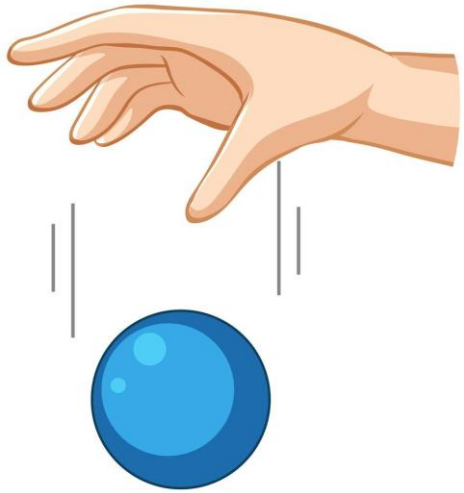
Dimensionality reduction

- Typical strategies for dimensionality reduction:
  - **Feature dropping** – Eliminates features not useful for prediction
  - **Feature selection** – Select a subset of the original features, discarding the others, without creating new ones
  - **Feature extraction** – Transform the original features to create a new set of features, usually with lower dimensionality
  - **Clustering based reduction** – Group similar features and replace them with cluster representatives
  - **Autoencoders** – Use neural networks to learn compressed representations automatically

Feature engineering

Dimensionality reduction – Feature dropping

- **Feature dropping** – Eliminates features not useful for prediction
  - Features with no predictive value
  - Applied when issues cannot be fixed via transformation
  - Simplifies the dataset and reduces complexity
  - Helps avoid learning spurious patterns
  - Typically performed early in preprocessing



Feature engineering

Dimensionality reduction – Feature dropping

- Which kind of features usually get dropped?
  - **Inconsistent or poorly defined features**
    - Variables with unclear semantics or inconsistent data collection processes
    - Example: A field filled differently across departments without a standard definition
  - **Irrelevant Identifiers**
    - IDs that carry no predictive meaning
    - Example: Customer ID, Transaction ID
  - **Revealing *data leakage***
    - Feature containing information about the future or target information
    - Example: “Loan Approved Date” when predicting loan approval

Feature engineering

Dimensionality reduction – Feature dropping

- Which kind of features usually get dropped?
  - **High missingness**
    - Drop variables with >70% missing values
    - Example: “Secondary Phone Number” in CRM data
  - **Low variance (near-constant features)**
    - Feature has almost the same value for all records
    - Example: Country = “Portugal” in a national dataset
  - **Unstable / noisy signals**
    - Feature shows erratic behavior across time periods
    - Example: A temperature sensor in a factory fluctuates wildly from minute to minute due to calibration errors, making the readings unreliable for predictive modelling

Feature engineering

Dimensionality reduction – Feature selection

- **Feature selection** – Select a subset of the original features, discarding the others, without creating new ones
  - Removes redudant features
  - Ranks and prioritizes the most useful features
- Techniques:
  - **Filter methods** – evaluate features independently of the model
  - **Wrapper methods** – evaluate subsets of features by training models
  - **Embedded methods** – feature selection occurs during model training

Feature engineering

Dimensionality reduction – Feature selection: filter methods

- **Filter methods** – Evaluate features independently of any learning algorithm and decide which features to keep or remove based on statistical properties

- Typical techniques:
  - **Correlation / Covariance Analysis** – Remove highly correlated numerical features
  - **Multicollinearity Detection (VIF)** – Identify linear dependence between numerical features
  - **Mutual Information** – Quantify dependency between categorical features and target variable
  - **Chi-square Test** – Evaluate categorical features' association with target

Feature engineering

Dimensionality reduction – Feature selection: filter methods

Covariance

$$Cov(X_1, X_2) = \frac{1}{N}\sum_{i=1}^{N}(x_{1i} - \mu_1)(x_{2i} - \mu_2)$$

- **Correlation / Covariance Analysis** – Remove highly correlated numerical features

Feature 1	Feature 2	Correlation	Action	Consequence / Explanation
Height (cm)	**Height (inches)**	0.99	Remove one	The two features convey identical information Removing one reduces redundancy without losing information
Temperature (C)	**Temperature (F)**	1.00	Remove one	Perfect correlation; one feature is enough Saves computational cost
Age	**Years of Experience**	0.20	Keep both	Low correlation means features provide independent information No action needed

- Highly correlated features do not add new information, so removing one simplifies the dataset
- Low correlation means features are independent, so both are useful

Feature engineering

Dimensionality reduction – Feature selection: filter methods

Variance Inflaction Factor

$$VIF_i = \frac{1}{1 - R_i^2}$$

- **Multicollinearity Detection (VIF)** – Identify linear dependence between numerical features

Feature	VIF	Action	Consequence / Explanation
Weight	12	Remove	High VIF (> 10) indicates strong linear dependence with other features Removing avoids inflated variance in regression coefficients
Height	8	Keep	Moderate VIF]5;10]; may be acceptable depending on model
Education Level	2	Keep	Low VIF [1;5]; features are essentially independent

- Multicollinearity occurs when one feature can be predicted from others
- Removing or combining features stabilizes the model and improves interpretability

Feature engineering

Dimensionality reduction – Feature selection: filter methods

Mutual information

$$\mathrm{MI}(X;Y) = \sum_{x \in X} \sum_{y \in Y} \frac{n(x,y)}{N} \log \frac{N \cdot n(x,y)}{n(x)\, n(y)}$$

- **Mutual Information** – Quantify dependency between categorical features and target variable

Feature	Target	MI with target	Action	Consequence / Explanation
Attendance Rate	**Student success**	0.45	Keep	Strong dependency on target. Retaining improves predictive performance.
Homework Completion	**Student success**	0.12	Maybe remove	Weak dependency. May contribute little, possibly adds noise.
Favorite Subject	**Student success**	0.01	Remove	Very low dependency; does not help predict target. Removing reduces noise and model complexity.

- Mutual information measures how much knowing the feature reduces uncertainty about the target
- Features with high MI are informative; low MI features can be safely discarded.

Feature engineering

Dimensionality reduction – Feature selection: filter methods

Chi-square

$$\chi^2 = \sum_i \sum_j \frac{(O_{ij} - E_{ij})^2}{E_{ij}}$$

- **Chi-square Test** – Evaluate categorical features' association with target

Feature	Target	Chi-square p-value	Action	Consequence / Explanation
Marital Status	**Loan Default**	0.800	Remove	High p-value → weak association. Feature is not useful for prediction.
Credit History	**Loan Default**	0.001	Keep	Low p-value → strong association. Feature is predictive and retained.
Occupation	**Loan Default**	0.050	Maybe keep	Borderline significance; may be useful in combination with other features.

- Chi-square tests whether a categorical feature is independent of the target
- High p-values → no predictive value; low p-values → informative and retained

Feature engineering

Dimensionality reduction – Feature selection: wrapper methods

- **Wrapper methods** – evaluate subsets of features by training models
  - Wrapper methods select subsets of features based on model performance
  - They repeatedly train a model using different feature combinations and evaluate results
  - Goal: keep features that improve model performance, discard the others
  - Example:

Feature	Description
x_1	Attendance rate
x_2	Homework completion
x_3	Favorite subject
x_4	Hours studied per week
y	Pass/Fail

Feature engineering

Dimensionality reduction – Feature selection: wrapper methods

- **Forward selection**
  - Start with no features
  - Add each feature individually and train a model
  - Select the feature that gives the best performance

Step	Feature added	Model Accuracy	Action
1	x_1	0.70	Keep x_1
2	x_4	0.85	Keep x_4
3	x_2	0.87	Keep x_2
4	x_3	0.87	No improvement → discard x_3

Feature engineering

Dimensionality reduction – Feature selection: wrapper methods

- **Backward elimination**
  - Start with all features
  - Remove one at a time
  - Stop removing when performance drops significantly

Step	Features in model	Feature removed	Model accuracy	Consequence / Explanation
1	x_1, x_2, x_3, x_4	—	0.87	Starting model with all features
2	x_1, x_2, x_3, x_4	x_4	0.87	Removing x_4 does not reduce accuracy → x_4 was irrelevant
3	x_1, x_2, x_3	x_3	0.86	Accuracy drops slightly → x_3 contributes a little; may decide to keep or remove
4	x_1, x_2	—	0.86	No further improvement possible; final subset = $\{x_1, x_2\}$

Feature engineering

Dimensionality reduction – Feature selection: embedded methods

- **Embedded methods** – Feature selection occurs during model training
  - The learning algorithm itself decides which features to keep or discard
  - Often uses regularization or tree-based models to identify **important features**
- Considers feature interactions automatically
- Less computationally expensive than wrapper methods (no repeated training on subsets)
- Produces compact, high-performing feature sets

Feature engineering

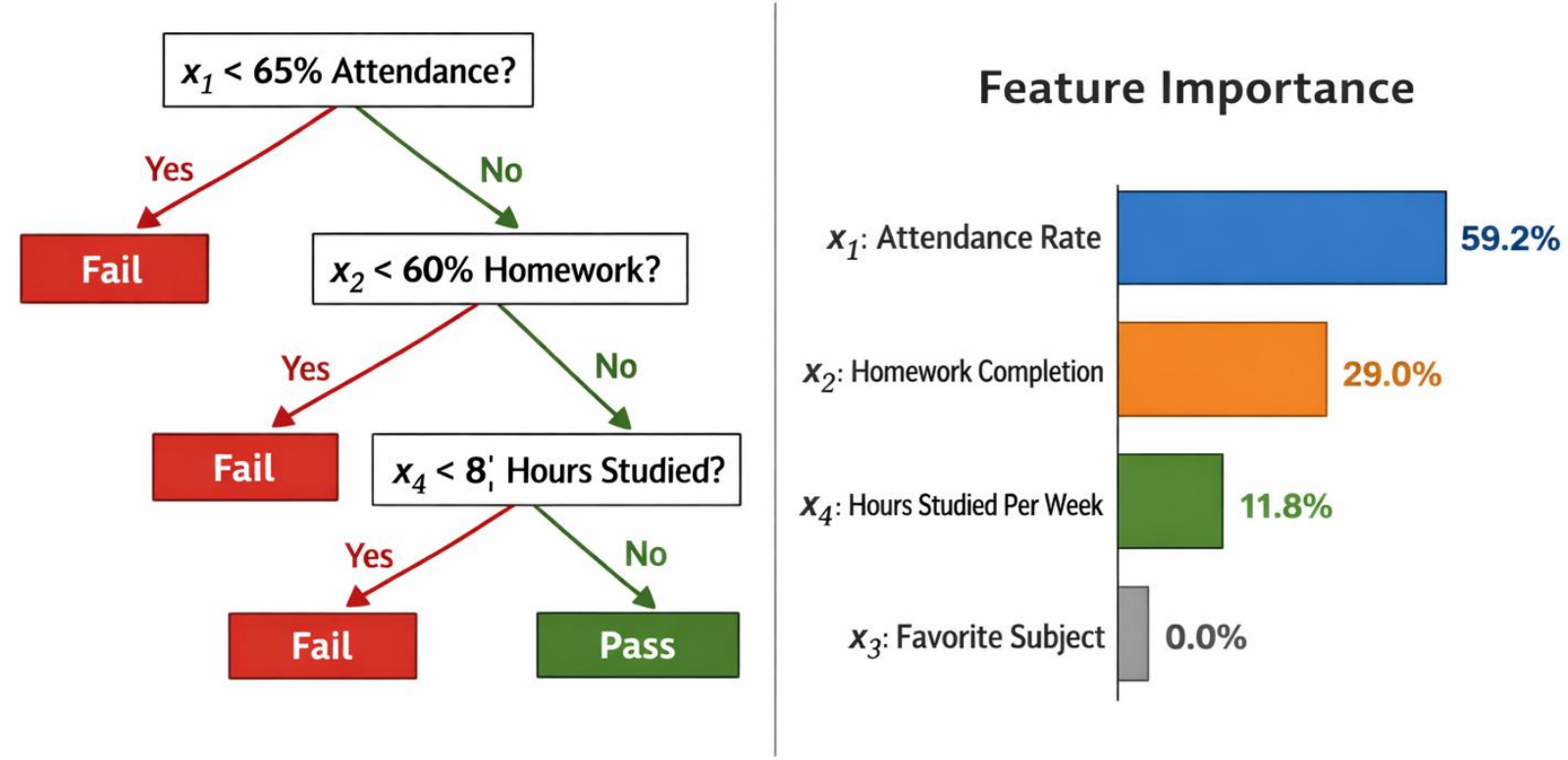
Dimensionality reduction – Feature selection: embedded methods

- Example of embedded method with **decision trees** and **feature importance**
  - Trees split the dataset based on features that
    - **Best separate the target variable classes** (classification)
    - **Reduce target variable variance** (regression)
  - The model can calculate feature importance based on:
    - How much the feature reduces impurity (e.g., Gini, entropy) across all splits
    - How often the feature is used in splits
  - Features with zero or very low importance can be removed

Feature engineering

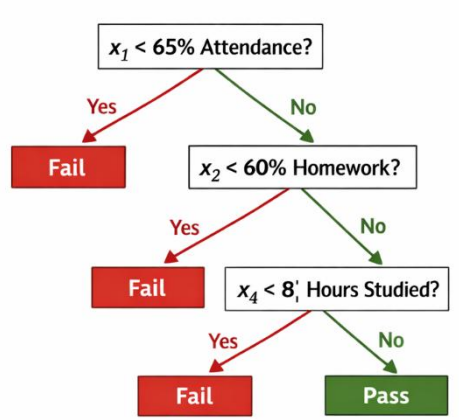
Dimensionality reduction – Feature selection: embedded methods

- Example of embedded method with **decision trees** and **feature importance**



Feature engineering

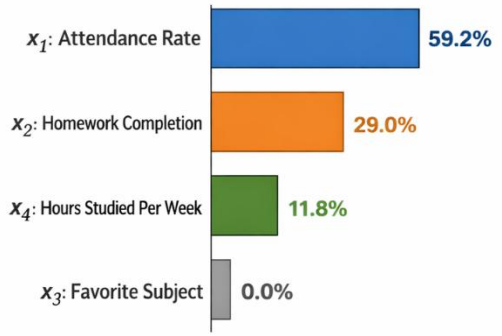
Dimensionality reduction – Feature selection: embedded methods



- Example of embedded method with **decision trees** and **feature importance**
  - Assuming a dataset with 1000 students

Feature	Node position	Examples used	Information gain	Contribution
x_1	Root	1000	0.50	0.50 × 1000 = 500
x_2	Level 2	700	0.35	0.35 × 700 = 245
x_4	Level 3	400	0.25	0.25 × 400 = 100
x_3	—	—	0.00	0

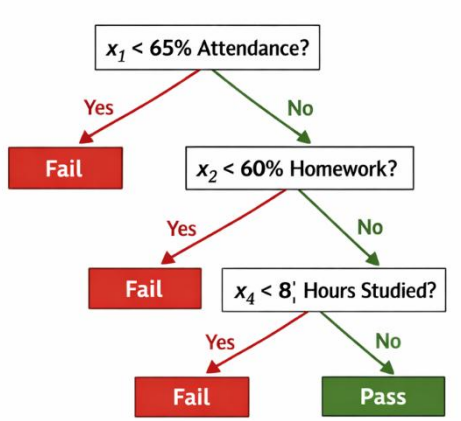
Feature	Importance (%)
x_1 = Attendance rate	59.2%
x_2 = Homework completion	29.0%
x_4 = Hours studied per week	11.8%
x_3 = Favorite subject	0%

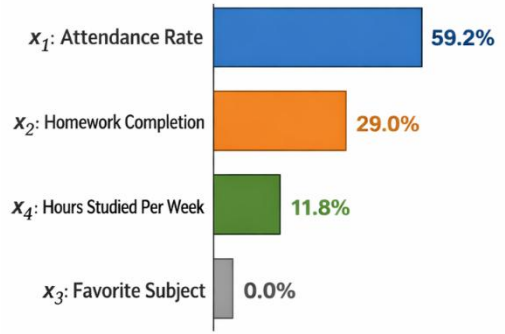


Feature engineering

Dimensionality reduction – Feature selection: embedded methods

- Example of embedded method with **decision trees** and **feature importance**
  - Interpretation:
    - x_1 = Attendance rate – dominant predictor
    - x_2 = Homework completion – provides strong additional separation
    - x_4 = Hours studied per week – refines the decision boundary
    - x_3 = Favorite subject – no contribution
  - **C5.0** performs feature selection during training
    - At each node, only the feature with the highest information gain is selected
    - Irrelevant features are naturally excluded





Feature engineering

Dimensionality reduction – Feature extraction

- **Feature extraction** – Transform the original features to create a new set of features, usually with lower dimensionality

- Techniques:
  - **Linear methods** – Creates new features as linear combinations of the original features, preserving most of the variance or class-separating information
  - **Non-linear methods** – Creates new features through non-linear transformations that capture complex structures in the data, often revealing patterns not visible in the original decision space

Feature engineering

Dimensionality reduction – Feature extraction: linear methods

- **Linear methods** – Creates new features as linear combinations of the original features, preserving most of the variance or class-separating information
- Simplifies the data, often improving computational efficiency and helping models generalize better
- Techniques:
  - **Principal Component Analysis (PCA)** – focuses on capturing the directions of maximum variance in the data
  - **Linear Discriminant Analysis (LDA)** – focuses on finding linear combinations that best separate different classes
- There is usually **loss of interpretability**

Feature engineering

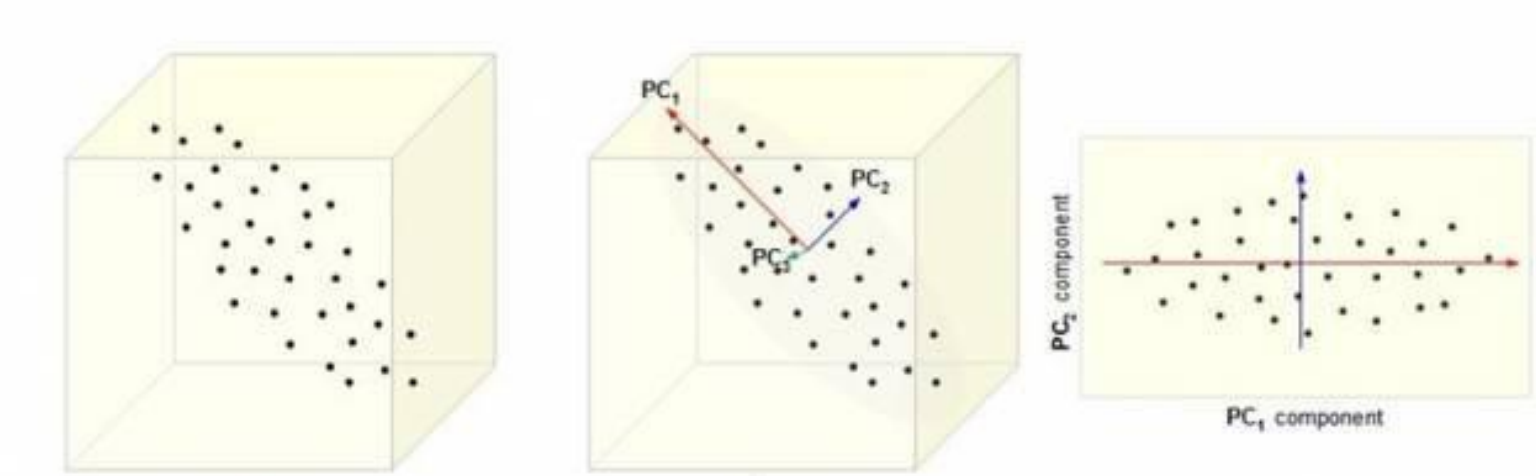
Dimensionality reduction – Feature extraction: linear methods

- **Principal Component Analysis (PCA)** – focuses on capturing the directions of maximum variance in the data
  - Goal: Transform the original features into a smaller set of **principal components** (new features) that retain most of the information
  - The first component captures most variance; subsequent components capture remaining variance
  - Steps:
    1. Center the data (subtract feature means)
    2. Compute the covariance matrix
    3. Eigen decomposition to find principal components
    4. Project data onto top k components

Feature engineering

Dimensionality reduction – Feature extraction: linear methods

- **Principal Component Analysis (PCA)** – focuses on capturing the directions of maximum variance in the data



https://www.linkedin.com/pulse/principal-component-analysis-utkarsh-sharma/

Feature engineering

Dimensionality reduction – Feature extraction: linear methods

- **Principal Component Analysis (PCA)**
  - Example: measure student engagement

Feature	Description
x_1	Attendance hours
x_2	Hours studied per week
x_3	Homework score

Student	x_1	x_2	x_3
A	2	5	3
B	3	6	4
C	4	8	5
D	5	9	7
E	6	11	8

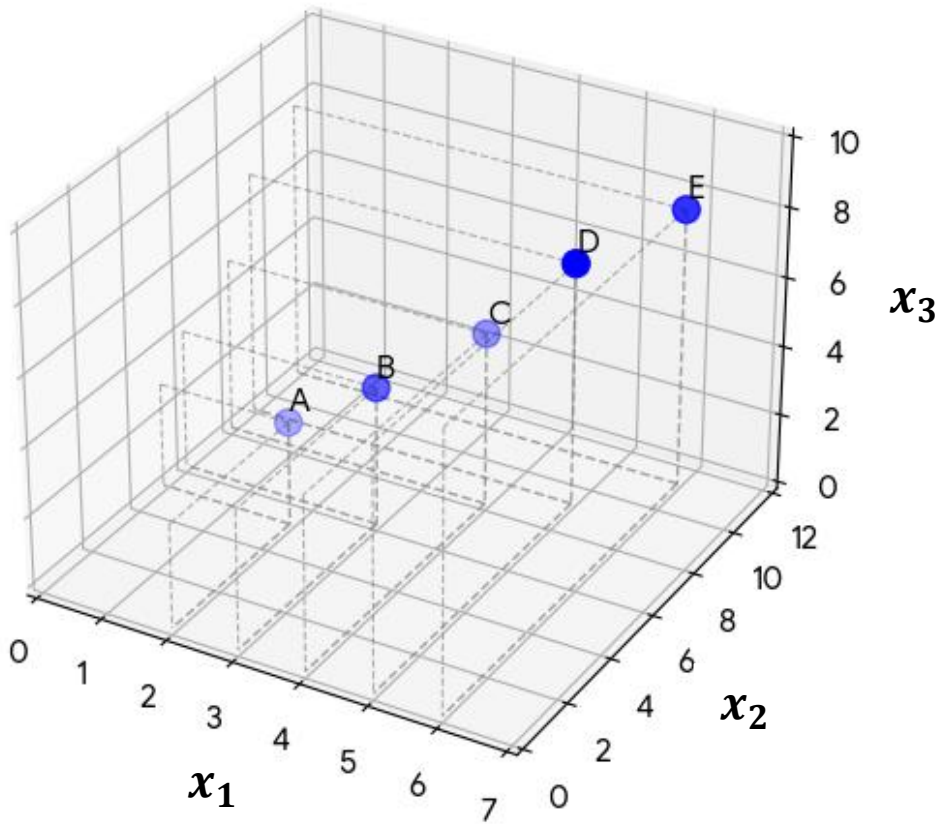
  - Study and attendance tend to increase together
  - Homework increases as well, but not perfectly
  - So, the data is not perfectly linear

Feature engineering

Dimensionality reduction – Feature extraction: linear methods

- **Principal Component Analysis (PCA)**
  - Example: measure student engagement

Student	x_1	x_2	x_3
A	2	5	3
B	3	6	4
C	4	8	5
D	5	9	7
E	6	11	8



Feature engineering

Dimensionality reduction – Feature extraction: linear methods

- **Principal Component Analysis (PCA)**

  1. Center the data (subtract feature means)

Feature	μ
x_1	4.0
x_2	7.8
x_3	5.4

$$x_{iC} = x_i - \mu_i$$

Original data

Student	x_1	x_2	x_3
A	2	5	3
B	3	6	4
C	4	8	5
D	5	9	7
E	6	11	8

Centered data

Student	x_{1C}	x_{2C}	x_{3C}
A	-2.0	-2.8	-2.4
B	-1.0	-1.8	-1.4
C	0.0	0.2	-0.4
D	1.0	1.2	1.6
E	2.0	3.2	2.6

Feature engineering

Dimensionality reduction – Feature extraction: linear methods

- **Principal Component Analysis (PCA)**

  2. Compute the covariance matrix

$$Cov(x_i, x_j) = \frac{1}{N}\sum_{i=1}^{N}(x_{iC} - \mu_i)(x_{jC} - \mu_j)$$

Cov	x_{1C}	x_{2C}	x_{3C}
x_{1C}	2.50	3.75	3.25
x_{2C}	3.75	5.70	4.85
x_{3C}	3.25	4.85	4.30

  - All covariances are positive → features are strongly correlated
  - x_2 has the highest variance → most spread
  - High redundancy → good candidate for dimensionality reduction with PCA

Feature engineering

Dimensionality reduction – Feature extraction: linear methods

- **Principal Component Analysis (PCA)**

  3. Eigen decomposition to find principal components

     - Solving $|Cov - \lambda I| = 0$
     - The eigenvalues for each principal component are:

Component	Eigenvalue	Explained variance
PC_1	$\lambda_1 = 12.35$	98.8%
PC_2	$\lambda_2 = 0.13$	1.0%
PC_3	$\lambda_3 = 0.02$	0.2%

     - Interpretation:
       - PC_1 captures most of the variation

Feature engineering

Dimensionality reduction – Feature extraction: linear methods

- **Principal Component Analysis (PCA)**

  3. Eigen decomposition to find principal components

     - Eigenvectors (principal directions):

$$v_1 = \begin{bmatrix} 0.45 \\ 0.67 \\ 0.59 \end{bmatrix}; \quad v_2 = \begin{bmatrix} -0.79 \\ 0.09 \\ 0.61 \end{bmatrix}; \quad v_3 = \begin{bmatrix} 0.41 \\ -0.74 \\ 0.53 \end{bmatrix}$$

     - As PC_1 dominates → almost all variance lies in a single direction
     - Strong redundancy among features
     - The data is effectively 1-dimensional

$$z_1 = 0.45x_1 + 0.67x_2 + 0.59x_3$$

Feature engineering

Dimensionality reduction – Feature extraction: linear methods

- **Principal Component Analysis (PCA)**

4. Project data onto top k components

$$z_1 = 0.45x_1 + 0.67x_2 + 0.59x_3$$

Original data

Student	x_1	x_2	x_3
A	2	5	3
B	3	6	4
C	4	8	5
D	5	9	7
E	6	11	8

Data projected in principal component

Student	z_1
A	6.02
B	7.73
C	10.11
D	12.41
E	14.79

Feature engineering

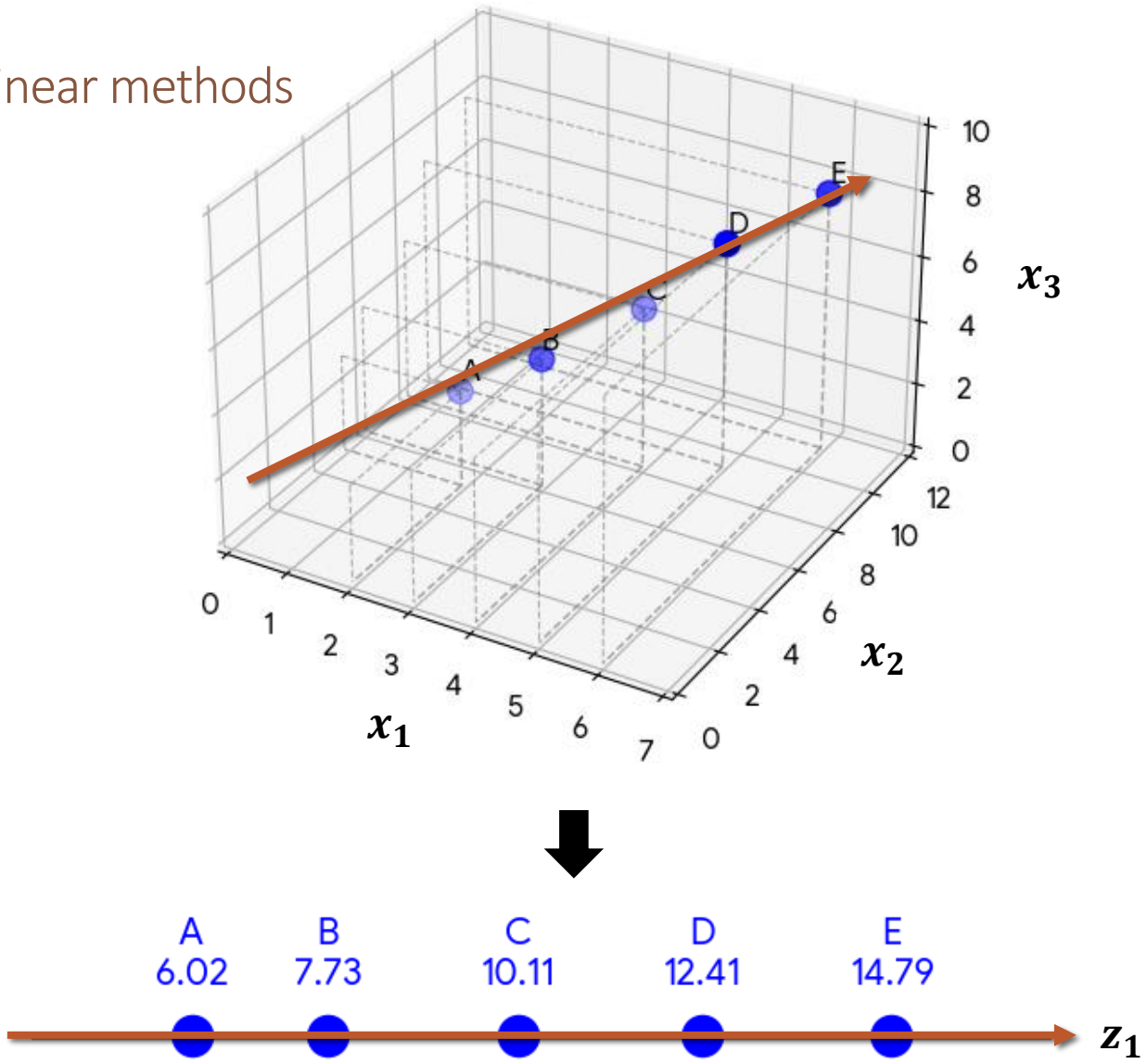
Dimensionality reduction – Feature extraction: linear methods

- **Principal Component Analysis (PCA)**

  4. Project data onto top k components

Data projected in principal component

Student	z_1
A	6.02
B	7.73
C	10.11
D	12.41
E	14.79



Feature engineering

Dimensionality reduction – Feature extraction: linear methods

- **Linear Discriminant Analysis (LDA)** – focuses on finding linear combinations that best separate different classes
  - Goal: Transform the original features into a smaller set of new variables, **discriminant components**, that maximize the separation between classes while keeping examples within the same class close together
  - The components are chosen to maximize the ratio between the variance between classes and the variance within classes
  - Steps:
    1. Compute the mean vector for each class
    2. Compute the within-class scatter matrix (variation inside each class)
    3. Compute the between-class scatter matrix (distance between class means)
    4. Solve the eigenvalue problem to obtain the discriminant directions
    5. Project the data onto the top k discriminant components

Feature engineering

Dimensionality reduction – Feature extraction: linear methods

- **Linear Discriminant Analysis (LDA)**
  - Example: measure student engagement

Feature	Description
x_1	Attendance hours
x_2	Hours studied per week
x_3	Homework score
y	Pass/fail course

Student	x_1	x_2	x_3	y
A	2	60	3	Fail
B	3	65	4	Fail
C	4	70	5	Fail
D	6	85	7	Pass
E	7	90	8	Pass
F	8	95	9	Pass

Feature engineering

Dimensionality reduction – Feature extraction: linear methods

- **Linear Discriminant Analysis (LDA)**

  1. Compute the mean vector for each class

     - Fail class

$$\mu_{Fail} = \begin{bmatrix} 3 \\ 65 \\ 4 \end{bmatrix}$$

     - Pass class

$$\mu_{Pass} = \begin{bmatrix} 7 \\ 90 \\ 8 \end{bmatrix}$$

     - Global mean:

$$\mu = \begin{bmatrix} 5 \\ 77.5 \\ 6 \end{bmatrix}$$

Student	x_1	x_2	x_3	y
A	2	60	3	Fail
B	3	65	4	Fail
C	4	70	5	Fail
D	6	85	7	Pass
E	7	90	8	Pass
F	8	95	9	Pass

Feature engineering

Dimensionality reduction – Feature extraction: linear methods

- **Linear Discriminant Analysis (LDA)**

  2. Compute the within-class scatter matrix (variation inside each class)

     - Measure spread inside each class:

$$S_W = \sum_{c \in classes} \sum_{x_i \in c} (x_i - \mu_c)(x_i - \mu_c)^T$$

     - After calculation:

$$S_W = \begin{bmatrix} 14 & 70 & 14 \\ 70 & 350 & 70 \\ 14 & 70 & 14 \end{bmatrix}$$

     - Interpretation: how much the students vary inside each class

Feature engineering

Dimensionality reduction – Feature extraction: linear methods

- **Linear Discriminant Analysis (LDA)**

  3. Compute the between-class scatter matrix (distance between class means)

     - Measure distance between class means:

$$S_B = \sum_{c \in classes} n_c(\mu_c - \mu)(\mu_c - \mu)^T$$

     - With $n_{Fail} = 3, n_{Pass} = 3$:

$$S_B = \begin{bmatrix} 48 & 300 & 48 \\ 300 & 1875 & 300 \\ 48 & 300 & 48 \end{bmatrix}$$

     - Interpretation: how far apart the class centers are

Feature engineering

Dimensionality reduction – Feature extraction: linear methods

- **Linear Discriminant Analysis (LDA)**

  4. Solve the eigenvalue problem to obtain the discriminant directions

     - Compute:

$$S_W^{-1} S_B v = \lambda v$$

     - The largest eigenvector gives the direction (linear combination of features) that best separates the classes
     - Other eigenvectors (if more than 2 classes) give additional discriminant axes
     - Example largest eigenvector (LD_1):

$$w = \begin{bmatrix} 0.04 \\ 0.25 \\ 0.04 \end{bmatrix}$$

Feature engineering

Dimensionality reduction – Feature extraction: linear methods

- **Linear Discriminant Analysis (LDA)**

5. Project the data onto the top k discriminant components

$$LD_1 = 0.04x_1 + 0.25x_2 + 0.04x_3$$

Original data

Student	x_1	x_2	x_3	y
A	2	60	3	Fail
B	3	65	4	Fail
C	4	70	5	Fail
D	6	85	7	Pass
E	7	90	8	Pass
F	8	95	9	Pass

Data projected in discriminant component

Student	LD_1	y
A	15.3	Fail
B	16.6	Fail
C	17.9	Fail
D	21.8	Pass
E	23.1	Pass
F	24.4	Pass

Feature engineering

Dimensionality reduction – Feature extraction: non-linear methods

- **Non-linear methods** – Creates new features through non-linear transformations that capture complex structures in the data, often revealing patterns not visible in the original decision space

- These transformations aim to:
  - Capture complex relationships between variables
  - Represent curved or manifold structures in the data
  - Reveal patterns that are not visible in the original feature space

- Difference in the transformation function

Linear

$$z = w_1 x_1 + w_2 x_2 + \cdots + w_p x_p$$

Non-linear

$$z = f(x_1, x_2, \ldots, x_p)$$

Feature engineering

Dimensionality reduction – Feature extraction: non-linear methods

- Patterns addressed by non-linear feature extraction methods
  - **Curved manifolds** – Data lie on curved surfaces that cannot be captured by linear projections
  - **Circular or radial patterns** – Relationships between variables form circular or radial structures
  - **Non-linearly separable clusters** – Groups are separated by complex, non-linear boundaries
  - **Low-dimensional manifolds** – High-dimensional data lie on an underlying lower-dimensional structure

Feature engineering

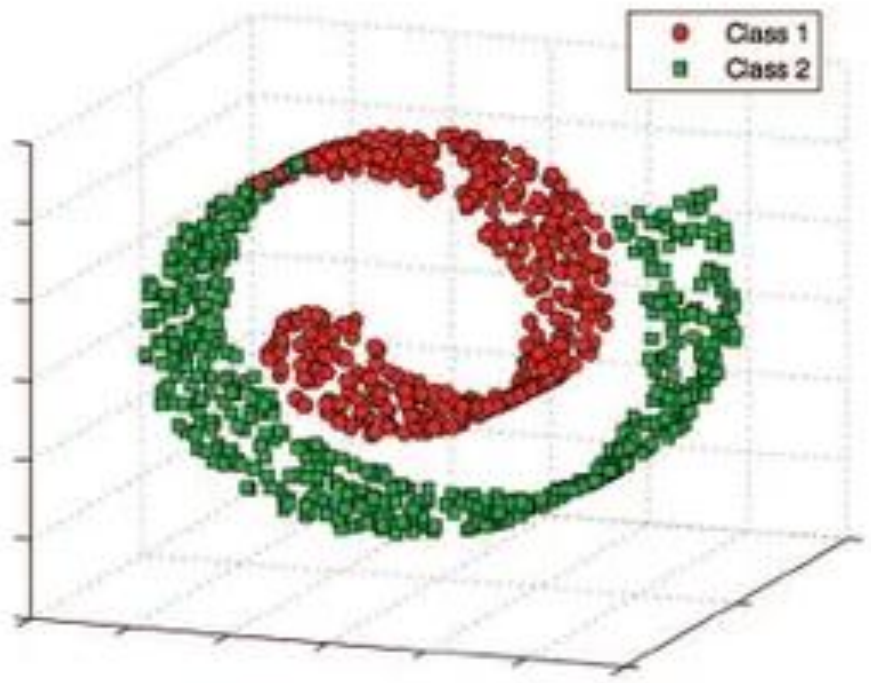
Dimensionality reduction – Feature extraction: non-linear methods

- Typical non-linear methods:
  - **Laplacian EigenMaps (LEM)**
  - **Locally Linear Embedding (LLE)**
  - **t-Distributed Stochastic Neighbor Embedding (t-SNE)**
  - Kernel PCA (Nonlinear extension of PCA)
  - Isomap (variation of LEM)
  - Uniform Manifold Approximation and Projection (UMAP)

Feature engineering

Dimensionality reduction – Feature extraction: non-linear methods

- The **Swiss roll** is a synthetic dataset commonly used to illustrate nonlinear structures in data

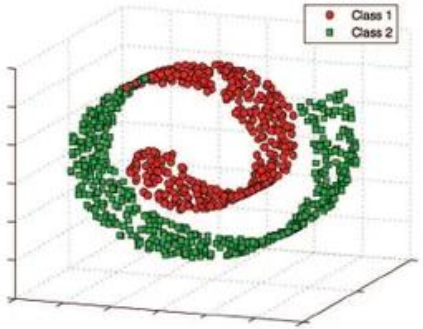


- Although the data lies in a three-dimensional space, the points lie on a two-dimensional curved surface (a rolled sheet)
- Two classes of points are shown:
  - **Red circles (Class 1)**
  - **Green squares (Class 2)**
- Along the surface of the roll, the two classes occupy different regions of the manifold.
- Even though the classes are separated along the surface, their Euclidean distances in 3D space **may appear small** because the roll folds different parts of the surface close to each other

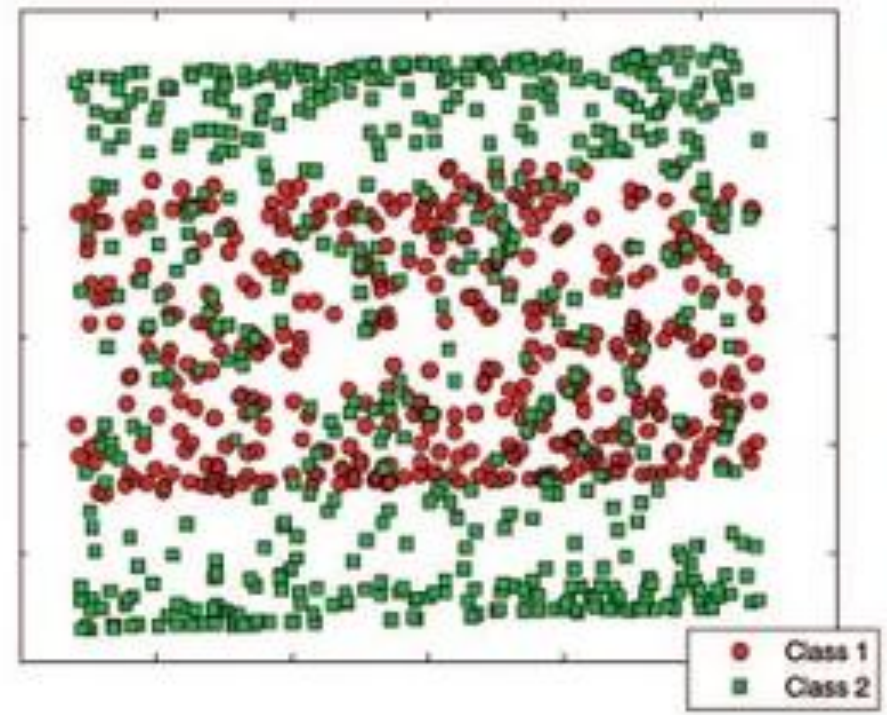
Swiss Roll And Swiss-Hole Reduction: https://scikit-learn.org/stable/auto_examples/manifold/plot_swissroll.html

Feature engineering

Dimensionality reduction – Feature extraction: non-linear methods



- **Linear Multidimensional Scaling (MDS) method**

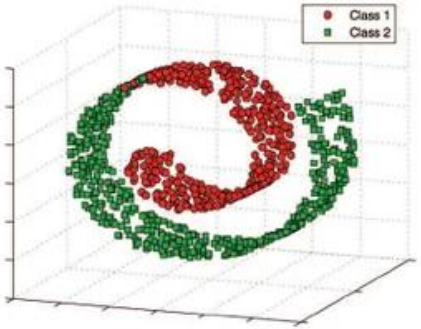


- Uses **Euclidean distance** between points
- Because the Swiss roll is folded, points from distant regions of the manifold can appear close in Euclidean space
- When projected to a lower dimension, this produces significant overlap between the two classes
- Linear methods **fail** because they assume the structure of the data is approximately **flat (linear)**

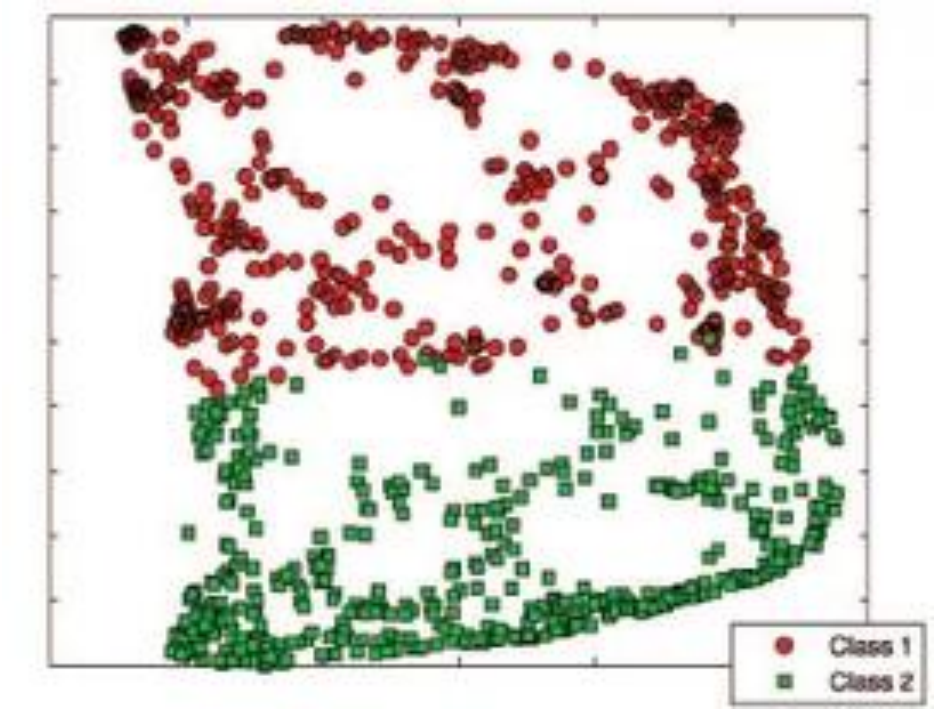
Swiss Roll And Swiss-Hole Reduction: https://scikit-learn.org/stable/auto_examples/manifold/plot_swissroll.html

Feature engineering

Dimensionality reduction – Feature extraction: non-linear methods



- **Laplacian EigenMaps (LEM) method**



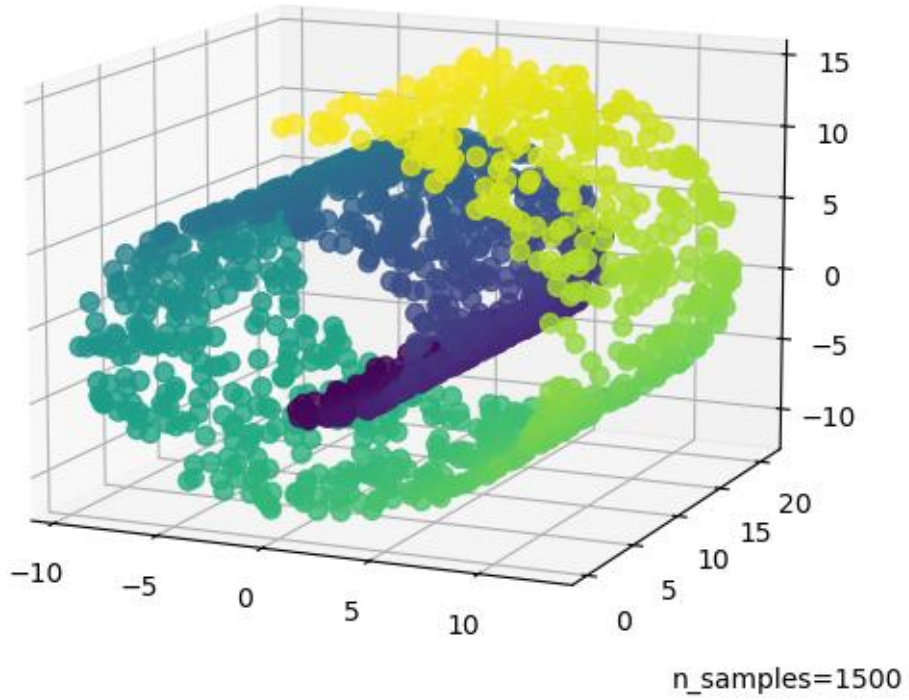
- A manifold learning technique
- Uses **geodesic distance**, i.e., distances measured along the surface of the manifold, not through the ambient space
- By respecting the manifold structure, the method unrolls the Swiss roll
- The resulting embedding separates the two classes **almost perfectly**

Swiss Roll And Swiss-Hole Reduction: https://scikit-learn.org/stable/auto_examples/manifold/plot_swissroll.html

Feature engineering

Dimensionality reduction – Feature extraction: non-linear methods

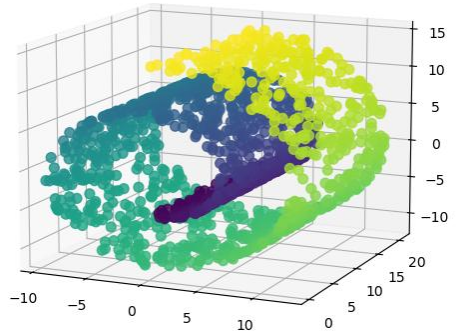
- Another example of methods to unroll the **Swiss roll**



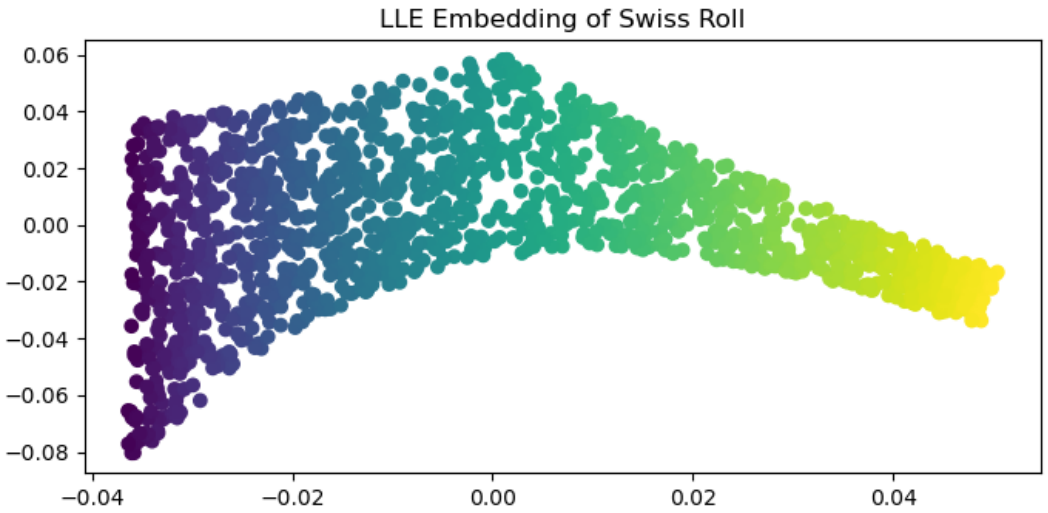
Investigating the Efficacy of Nonlinear Dimensionality Reduction Schemes in Classifying Gene- and Protein-Expression Studies: 10.1109/TCBB.2008.36

Feature engineering

Dimensionality reduction – Feature extraction: non-linear methods



- **Locally Linear Embedding (LLE) method**

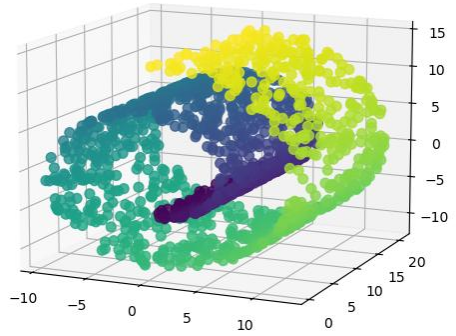


- LLE tries to preserve local linear relationships between points
- Each point is reconstructed as a linear combination of its neighbors in the high-dimensional space, and the same weights are used to place points in the lower-dimensional space
- LLE successfully “unrolls” the Swiss roll, preserving the smooth, continuous structure of the manifold
- LLE captures the global structure indirectly through local neighborhoods, making it effective for unfolding nonlinear manifolds

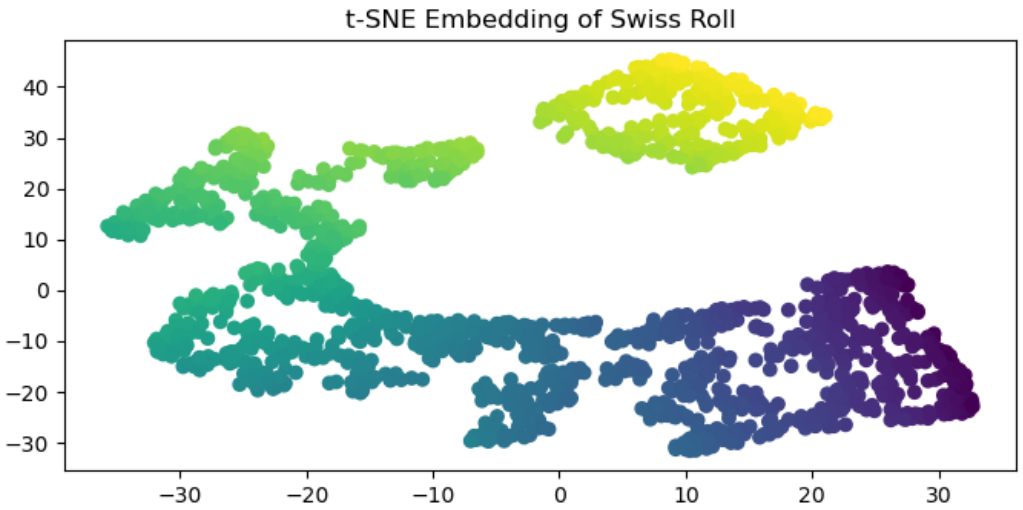
Investigating the Efficacy of Nonlinear Dimensionality Reduction Schemes in Classifying Gene- and Protein-Expression Studies: 10.1109/TCBB.2008.36

Feature engineering

Dimensionality reduction – Feature extraction: non-linear methods



- **t-Distributed Stochastic Neighbor Embedding (t-SNE) method**



- t-SNE focuses on preserving local similarities by converting distances into probabilities and trying to match these probabilities in the low-dimensional embedding
- t-SNE preserves the general shape of the data, meaning points that are close on the manifold remain close in the embedding
- It disrupts the continuous nature of the manifold, creating clusters or “clumps” that do not exist in the original data
- t-SNE is excellent for visualizing cluster structures, but it is not ideal for maintaining continuous trajectories along a manifold

Investigating the Efficacy of Nonlinear Dimensionality Reduction Schemes in Classifying Gene- and Protein-Expression Studies: 10.1109/TCBB.2008.36

Feature engineering

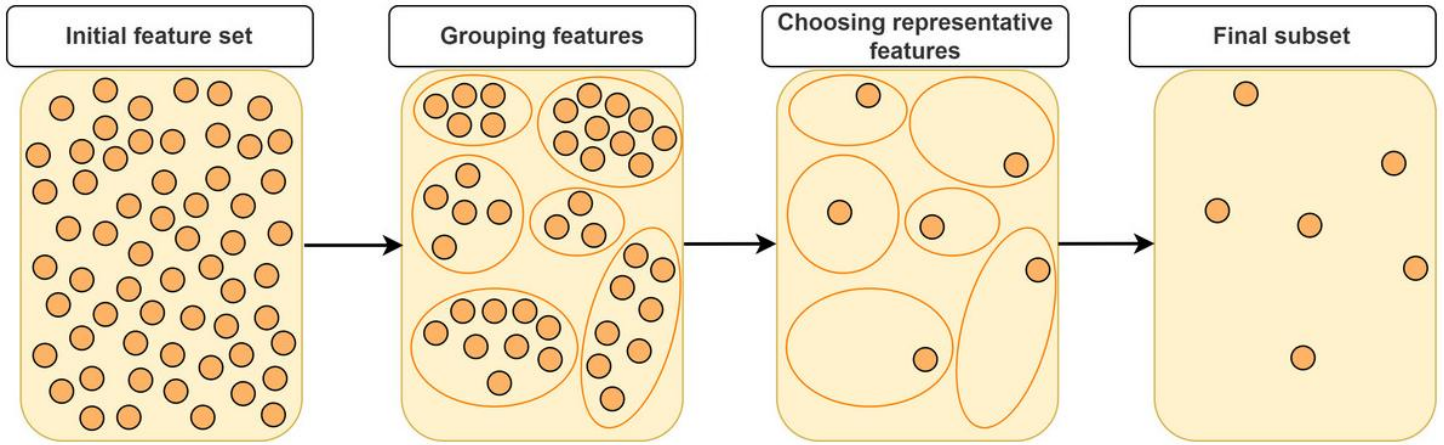
Dimensionality reduction – Feature extraction: non-linear methods

- Advantages
  - Captures complex structures
  - Reveals patterns hidden to linear methods
  - Useful for visualization and representation learning
- Limitations
  - Higher computational cost
  - Often sensitive to hyperparameters
  - Some methods do not provide an explicit transformation for new data
- Typical applications
  - Data visualization
  - Image and text representations
  - Preprocessing for machine learning models

Feature engineering

Dimensionality reduction – Clustering based reduction

- **Clustering based reduction** – Group similar features and replace them with cluster representatives
  - Features are often correlated (e.g., sensor readings, financial indicators)
  - The representative feature can be a cluster centroid, mean, or principal component
  - Reduces redundancy and dimensionality, keeping the main information



Review of feature selection approaches based on grouping of features: https://peerj.com/articles/15666/

Feature engineering

Dimensionality reduction – Clustering based reduction

- Example

ID	Sensor A	Sensor B	Sensor C	Sensor D
ID1	23.5	22.8	24.1	10.0
ID2	45.2	44.9	45.5	60.0
ID3	67.8	68.2	67.9	20.0

  - Correlation matrix between features

	Sensor A	Sensor B	Sensor C	Sensor D
Sensor A	1.000	0.999	0.999	0.100
Sensor B	0.999	1.000	0.999	0.120
Sensor C	0.999	0.999	1.000	0.080
Sensor D	0.100	0.120	0.080	1.000

Feature engineering

Dimensionality reduction – Clustering based reduction

- Example
  - Sensors A, B, C are highly correlated → grouped into Cluster 1
  - Sensor D is independent → forms its own Cluster 2
  - Cluster 1 values is the mean of features A, B and C for each example

ID	Mean(A,B,C)
ID1	(23.5+22.8+24.1)/3 ≈ 23.50
ID2	(45.2+44.9+45.5)/3 ≈ 45.20
ID3	(67.8+68.2+67.9)/3 ≈ 67.97

  - Reduced dimensionality dataset:

ID	Cluster 1 (A,B,C)	Cluster 2 (D)
ID1	23.50	10.00
ID2	45.20	60.00
ID3	67.97	20.00

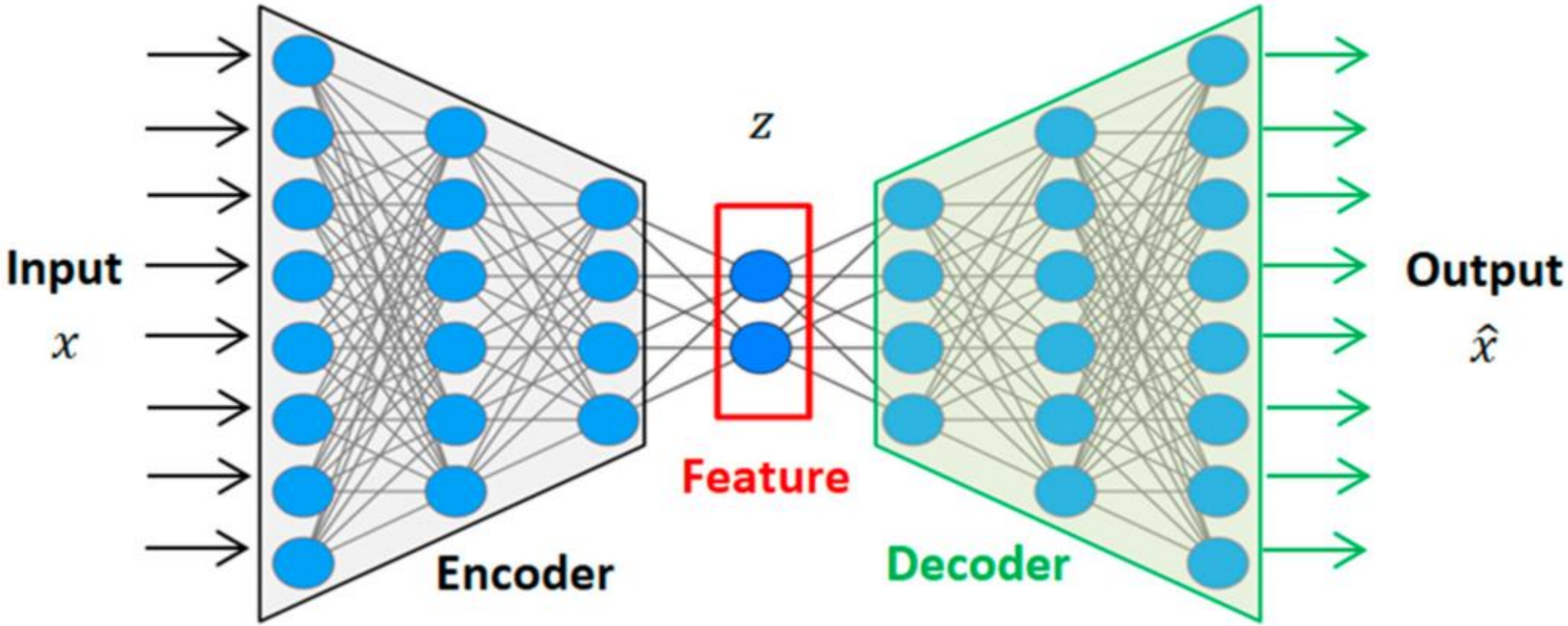
Feature engineering

Dimensionality reduction – Autoencoders

- **Autoencoders** – Use neural networks to learn compressed representations automatically
- Consist of:
  - **Encoder**: maps input → lower-dimensional latent space
  - **Decoder**: reconstructs input from latent representation
- The network learns features that capture the most relevant structure for reconstruction
  - Learns non-linear transformations, unlike PCA
  - Can capture complex dependencies between features
  - Creates a **latent space** which acts as reduced-dimensional feature representation

Feature engineering

Dimensionality reduction – Autoencoders



Neural Encoders: From Autoencoders to Modern Retrieval Architectures: https://muditb.com/neural-encoders-from-autoencoders-to-modern-retrieval-architectures/

Feature engineering

Dimensionality reduction – Autoencoders

- Example

Input space

ID	x_1	x_2	x_3	x_4
S1	0.5	0.2	0.1	0.7
S2	0.9	0.4	0.3	0.8
S3	0.2	0.1	0.2	0.3
S4	0.7	0.5	0.6	0.9
S5	0.3	0.3	0.4	0.5

encoder

Latent space

ID	z_1	z_2
S1	0.35	0.45
S2	0.65	0.55
S3	0.15	0.20
S4	0.70	0.75
S5	0.25	0.35

decoder

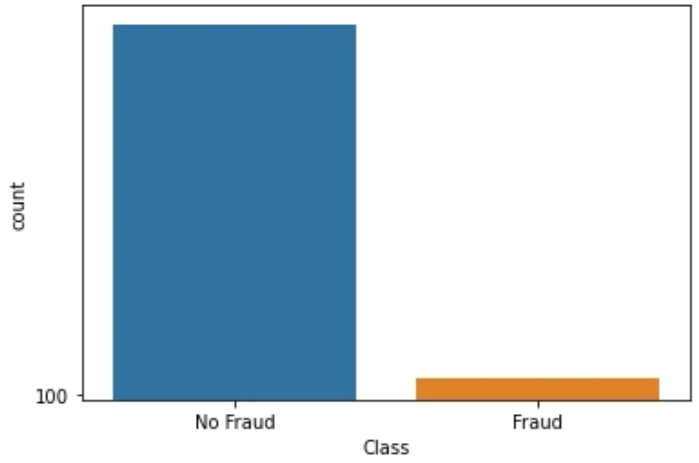
Output space

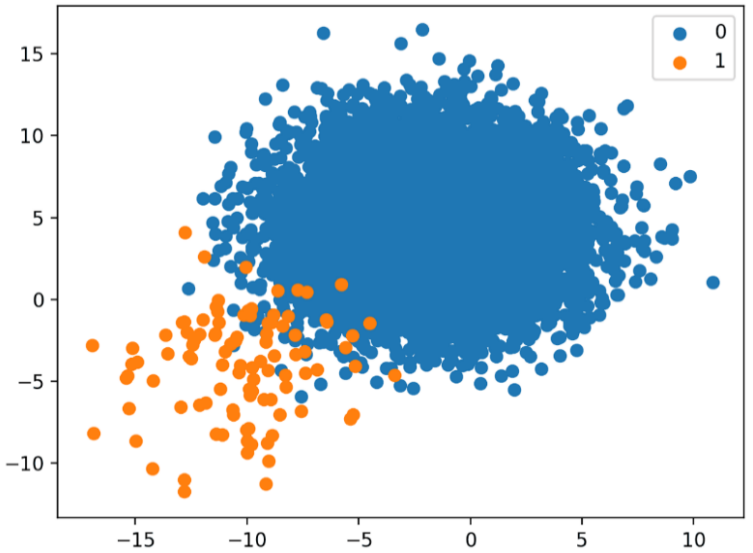
ID	x_1	x_2	x_3	x_4
S1	0.51	0.21	0.11	0.69
S2	0.88	0.42	0.32	0.81
S3	0.19	0.12	0.18	0.31
S4	0.71	0.53	0.61	0.88
S5	0.28	0.33	0.39	0.52

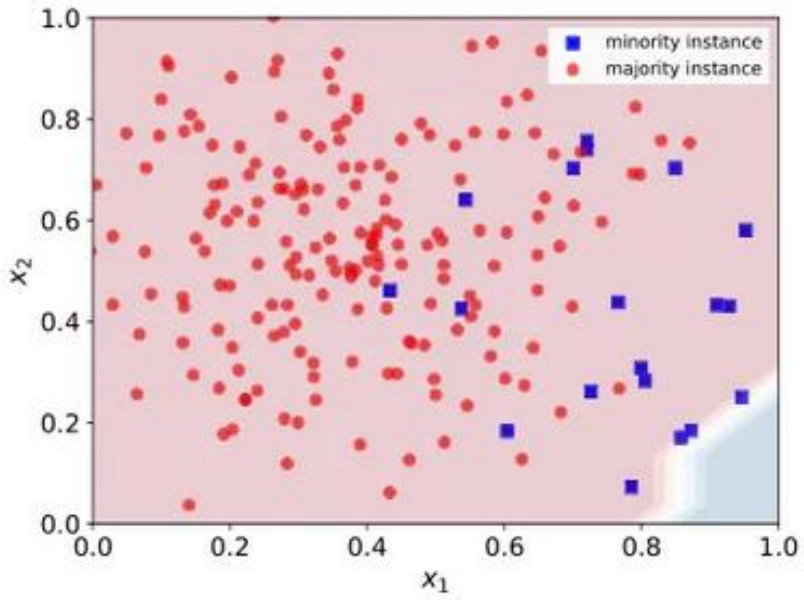
Feature engineering

Handling class imbalance

- **Handling class imbalance** is recommended to address situations where one class has significantly more examples than another in the target variable
- Examples include:
  - Fraud detection: 1% fraudulent transactions vs 99% normal
  - Churn prediction: 10% churners vs 90% non-churners







10 Techniques to Solve Imbalanced Classes in Machine Learning:
https://www.analyticsvidhya.com/blog/2020/07/10-techniques-to-deal-with-class-imbalance-in-machine-learning/

Feature engineering

Handling class imbalance

- Potential problems of class imbalance:
  - **Biased model predictions** – The model tends to predict the majority class more often, ignoring the minority class examples
  - **Poor performance metrics** – Accuracy can be misleading; a model predicting only the majority class can still show high accuracy
  - **Reduced sensitivity** – The model struggles to detect rare but important cases (low recall for minority class)
  - **Overfitting risk** – The model may overfit to the majority class patterns, ignoring the minority class
  - **Difficulty in learning** – Minority class patterns may be underrepresented, making it harder for algorithms to generalize
  - **Impact on decision making** – In applications like fraud detection or medical diagnosis, missing minority cases can have serious consequences

Feature engineering

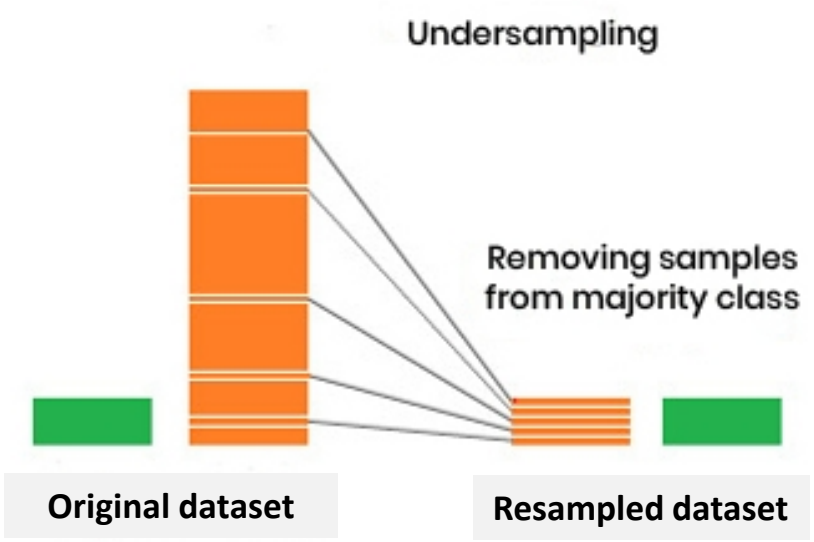
Handling class imbalance

- Typical strategies to handle class imbalance:
  - **Data-level approaches (resampling):**
    - **Oversampling** – increases the number of examples in the minority class
    - **Undersampling** – removes examples from the majority class
  - **Algorithm-level approaches:**
    - **Adjust class weights** – assign higher misclassification penalties to the minority class during model training
    - **Robust imbalance models** – apply algorithms that naturally handle unequal class distributions more effectively
  - **Hybrid approaches** – combine both resampling and algorithm adjustments

Feature engineering

Handling class imbalance – Data-level approaches (resampling)

- **Data-level approaches (resampling)**
  - Different strategies to achieve a balanced dataset



What is an Imbalanced Data in Data Mining?: https://www.janbasktraining.com/tutorials/imbalance-data-classification/

Feature engineering

Handling class imbalance – Data-level approaches (resampling): oversampling

- **Oversampling** – increases the number of examples in the minority class
  - Goal:
    - Reduce class imbalance
    - Allow the model to better learn patterns from the minority class
  - Main techniques:
    - **Random oversampling** – replicates existing minority examples
    - **Synthetic oversampling** – generates new artificial examples
      - Example of method: SMOTE *(Synthetic Minority Over-sampling TEchnique)*

Feature engineering

Handling class imbalance – Data-level approaches (resampling): oversampling

- **Random oversampling** – replicates existing minority examples
  - Balances an imbalanced dataset by randomly selecting and **replicating examples** from the minority class
  - Although this helps algorithms pay more attention to the minority class, it does not introduce new information and may increase the risk of overfitting

Class	Count
0 (majority)	900
1 (minority)	**100**

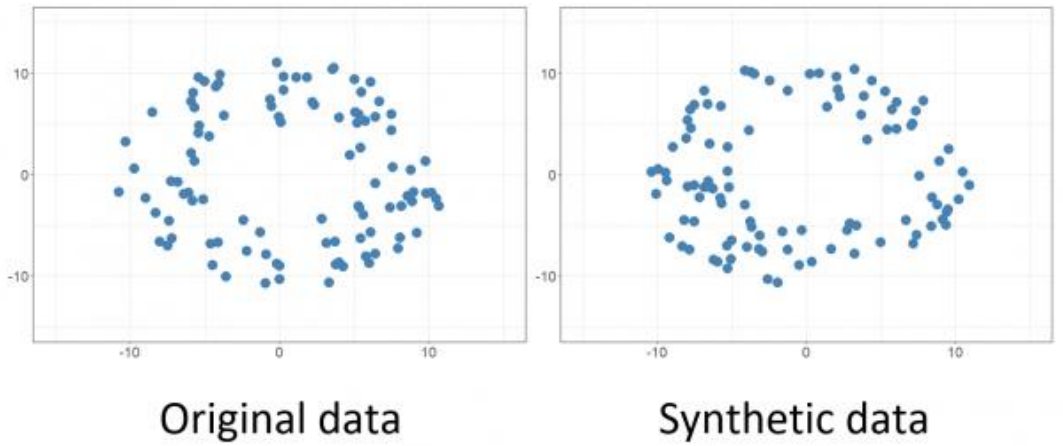
Class	Count
0	900
1	**900**

  - Since examples are selected **randomly with replacement**, some minority class examples may be replicated many times while others may not be replicated at all
    - This can cause the model to rely excessively on a small number of examples

Feature engineering

Handling class imbalance – Data-level approaches (resampling): oversampling

- **Synthetic oversampling** – generate new artificial examples
  - **Synthetic data** is data not derived from a real dataset
    - The new examples do not exist in the original dataset
    - They are artificially generated from existing minority examples
  - Some randomness is still involved (e.g., selecting the examples used to generate new ones)
  - Motivations:
    - Avoid excessive repetition of the same examples
    - Provide a richer representation of the minority class in the feature space



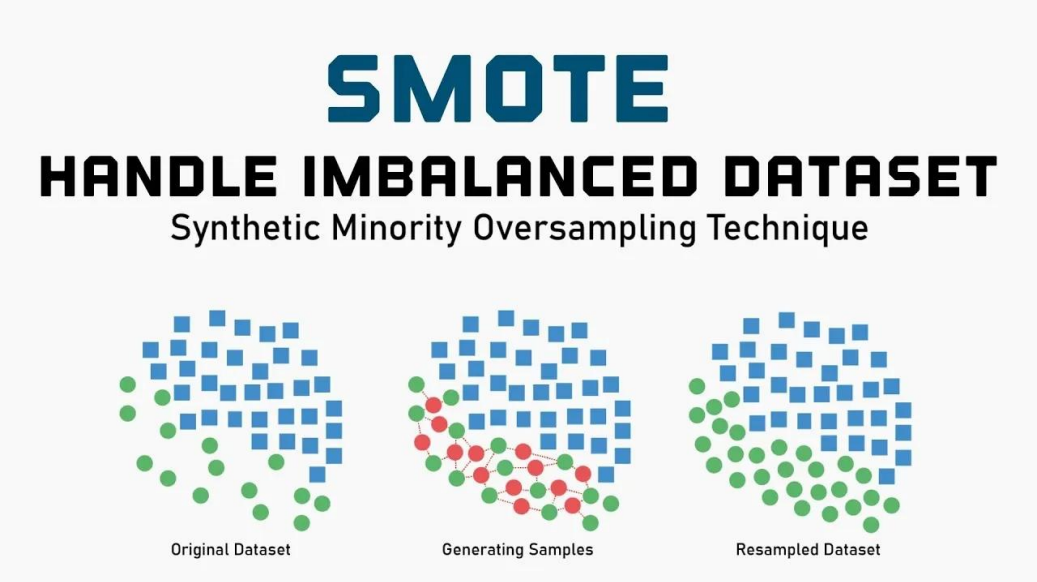
The synthetic data retains the structure of the original data but is not the same

Synthetic data: Unlocking the power of data and skills for machine learning
https://dataingovernment.blog.gov.uk/2020/08/20/synthetic-data-unlocking-the-power-of-data-and-skills-for-machine-learning/

Feature engineering

Handling class imbalance – Data-level approaches (resampling): oversampling

- **SMOTE *(Synthetic Minority Over-sampling TEchnique)***
  - Approach:
    - Select a minority class example
    - Find its k nearest neighbors (from the minority class)
    - Generate synthetic examples along the line connecting the example to its neighbors
  - New synthetic examples lie **between real examples**, increasing diversity
  - Reduces risk of overfitting compared to random oversampling



Balancing the Scales: How SMOTE Transforms Machine Learning with Imbalanced Data

https://python.plainenglish.io/balancing-the-scales-how-smote-transforms-machine-learning-with-imbalanced-data-a6d3367254cd

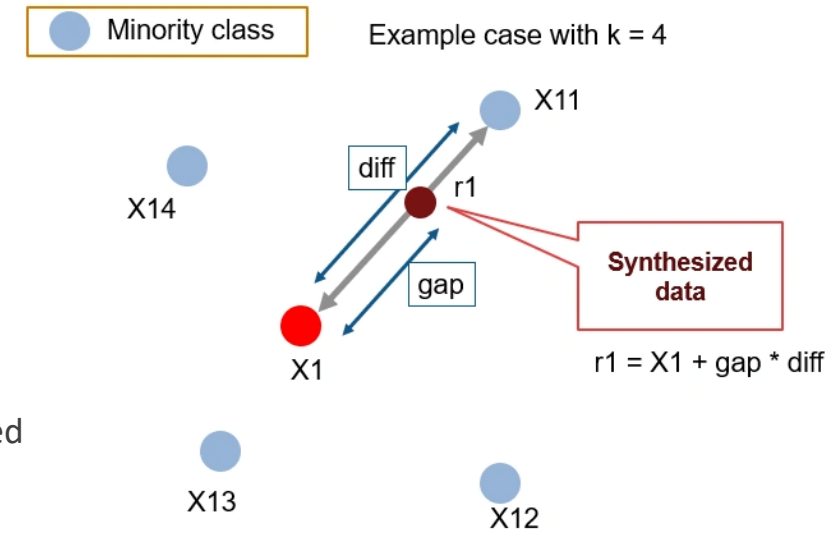
Feature engineering

Handling class imbalance – Data-level approaches (resampling): oversampling

- **SMOTE**
  1. Select a minority class example
     - Pick one example from the minority class to generate synthetic points
     - Example: a fraudulent transaction in a fraud detection dataset
  2. Find its nearest neighbors
     - Identify the k closest minority examples
     - These neighbors define the space where new synthetic examples can be generated
  3. Generate synthetic examples
     - Randomly choose one of its neighbors
     - Generate a new example along the line connecting the original example and the neighbor:

       $$v_{new} = v_{original} + gap(v_{neighbor} - v_{original})$$

       - gap is a random number between 0 and 1
       - This ensures the new examples lies between two real examples
  4. Repeat until the desired balance is reached
     - Continue generating synthetic examples until the minority class matches the majority class in size



SMOTE for Imbalanced Classification with Python

https://www.analyticsvidhya.com/blog/2020/10/overcoming-class-imbalance-using-smote-techniques/

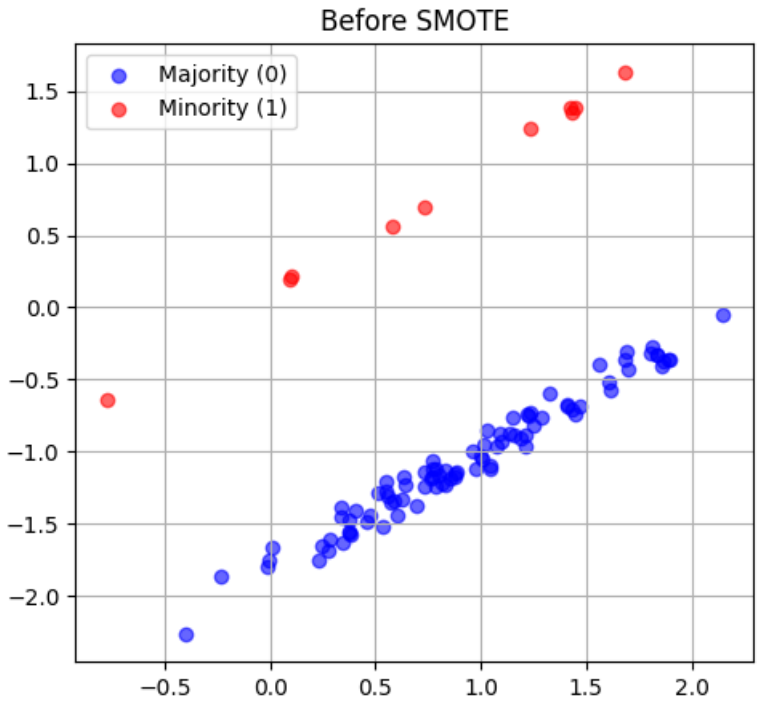
Feature engineering

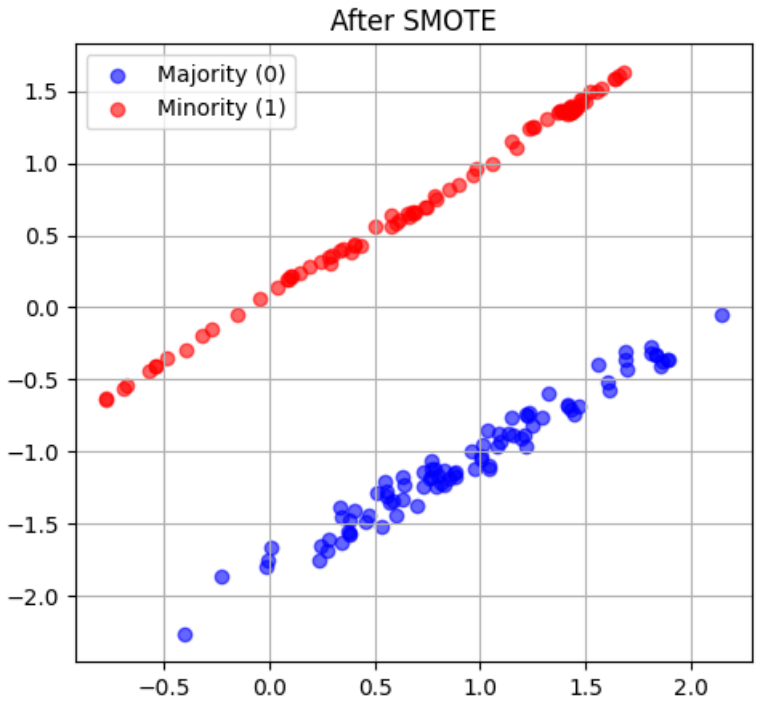
Handling class imbalance – Data-level approaches (resampling): oversampling

- **SMOTE**

Class	Count
0 (majority)	90
1 (minority)	**10**

Class	Count
0	90
1	**90**





Feature engineering

Handling class imbalance – Data-level approaches (resampling): undersampling

- **Undersampling** – removes examples from the majority class
  - Goal:
    - Balance the dataset without generating new examples
    - Allow the model to better learn patterns from the minority class
  - Advantages:
    - Simple to implement
    - Instance selection technique
    - Reduces dataset size → faster training
  - Limitations:
    - Risk of discarding useful information
    - Might affect minority class decision boundary if overdone

Feature engineering

Handling class imbalance – Data-level approaches (resampling): undersampling

- **Undersampling**
  1. Identify majority class examples
     - Example: Normal transactions in fraud detection
  2. Decide target size for majority class
     - Typically match minority class size or a defined ratio
  3. Randomly remove majority class examples
     - Optionally, use informative undersampling: keep examples near a minority cluster
  4. Resulting dataset
     - Continue removing examples until the minority class matches the majority class in size

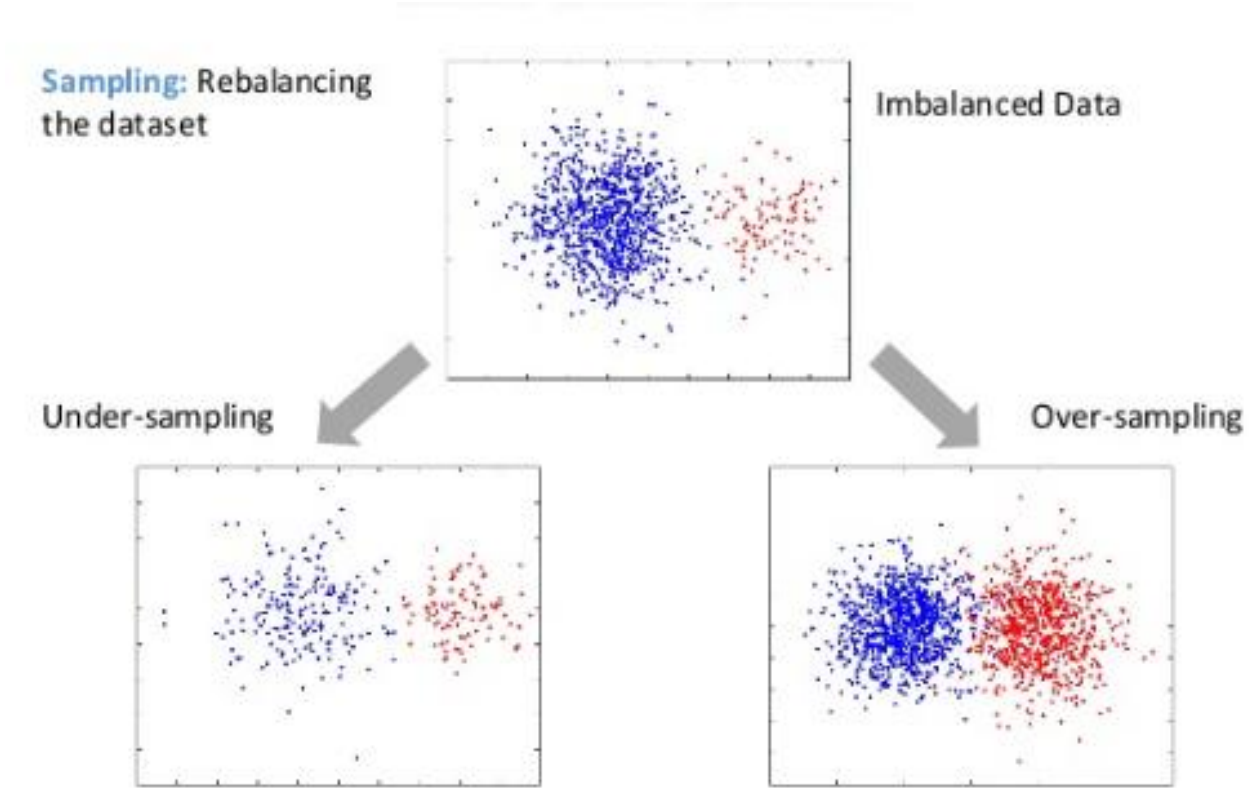
Class	Number of transactions
0 – Normal (majority)	900
1 – Fraud (minority)	100

Class	Number of transactions
0 – Normal	100
1 – Fraud	100

Feature engineering

Handling class imbalance – Data-level approaches (resampling)

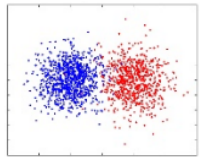
- **Which one to use?**

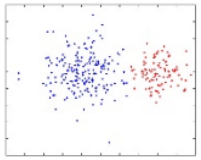


Feature engineering

Handling class imbalance – Data-level approaches (resampling)

- **Which one to use?**





Factor	Oversampling	Undersampling
Dataset size	Small or moderate (cannot afford to lose data)	Large (removing examples won't hurt)
Risk of overfitting	Moderate to high if replicating; Reduced with SMOTE	Low (removes duplicates)
Information loss	None (generates new examples)	Possible (removes majority examples)
Computational cost	Higher (dataset grows)	Lower (dataset shrinks)
Ease of implementation	Moderate (SMOTE requires libraries)	Very simple (random removal)
Recommended for	Small, imbalanced datasets where minority class patterns are important	Large datasets where speed and memory are concerns

Feature engineering

Handling class imbalance – Algorithm-level approaches

- **Algorithm-level approaches**
  - Improve model training on imbalanced datasets by modifying the algorithm rather than the data
  - Common techniques:
    - **Adjust class weights** – Make the model “pay more attention” to minority classes
    - **Robust imbalance models** – Use algorithms inherently designed to handle skewed class distributions
  - These approaches aim to **reduce bias toward the majority class** and improve predictive performance for all classes

Feature engineering

Handling class imbalance – Algorithm-level approaches: adjust class weights

- **Adjust class weights** – assign higher misclassification penalties to the minority class during model training
  - Modify the learning algorithm instead of the dataset
  - Assign greater importance to errors on the minority class
  - Applicable to multiple learning algorithms:
    - Logistic Regression
    - Decision Trees / Random Forest
    - Support Vector Machines (SVM)
  - Advantages:
    - Dataset size remains unchanged
    - No duplication or removal of examples
    - Reduces overfitting risk

Feature engineering

Handling class imbalance – Algorithm-level approaches: adjust class weights

- Normally, all misclassifications are treated equally

Loss for classification

$$\gamma = \big(M, (X, y)\big) = I[y \neq \hat{y}]$$

Loss for regression

$$\gamma = \big(M, (X, y)\big) = (y - \hat{y})^2$$

- Using an **empirical average loss function**:

$$\text{Loss}(M) = \frac{1}{n} \sum_{i=1}^{n} \gamma\big(M, (X_i, y_i)\big)$$

it is possible to compute **performance metrics** to evaluate a model

Feature engineering

Handling class imbalance – Algorithm-level approaches: adjust class weights

- Changing the weights:

Class	Weight (w)
0 – Majority	1
1 – Minority	9

- Using an **empirical average weighted loss function**:

$$\text{WeightedLoss}(M) = \frac{1}{n}\sum_{i=1}^{n} w_i \times \gamma\big(M, (X_i, y_i)\big)$$

  - w_i = class weight of example i

- Misclassification of minority class examples is **penalized more heavily**, guiding the model to focus on these

Feature engineering

Handling class imbalance – Algorithm-level approaches: robust imbalance models

- **Robust imbalance models** – apply algorithms that naturally handle unequal class distributions more effectively
  - Instead of modifying the dataset, the model construction process itself makes the algorithm less sensitive to class imbalance.
  - Examples of techniques:
    - **Decision boundary optimisation** – The boundary is shaped by examples near it, not class counts
    - **Sequential error correction** – Misclassified examples, often minority, get more focus
    - **Local learning** – Minority examples influence predictions in their neighborhood
  - Advantages:
    - Dataset size remains unchanged
    - No synthetic data is introduced
    - Minority examples can still influence the model through the learning mechanism

Feature engineering

Handling class imbalance – Algorithm-level approaches: robust imbalance models

- **Decision boundary optimization**
  - The goal is to detect fraudulent transactions with classes: **Normal transaction** (majority), **Fraud** (minority)

Transaction	Amount	Risk Score	Type
T1	15	0.10	Normal
T2	25	0.15	Normal
T3	30	0.20	Normal
T4	45	0.30	Normal
T5	55	0.35	Normal
T6	65	0.40	Normal
T7	70	0.45	Normal
T8	75	0.50	Fraud
T9	85	0.70	Fraud
T10	95	0.90	Fraud

Class distribution:

- **Normal:** 7 examples
- **Fraud:** 3 examples

Feature engineering

Handling class imbalance – Algorithm-level approaches: robust imbalance models

- **Decision boundary optimization**
  - A boundary-based model (e.g. Logistic Regression or SVM) learns a separation at: **Risk Score ≈ 0.60**

Transaction	Risk Score	True Class	Prediction
T1	0.10	Normal	**Normal**
T2	0.15	Normal	**Normal**
T3	0.20	Normal	**Normal**
T4	0.30	Normal	**Normal**
T5	0.35	Normal	**Normal**
T6	0.40	Normal	**Normal**
T7	0.45	Normal	**Normal**
T8	0.50	Fraud	**Normal**
T9	0.70	Fraud	**Fraud**
T10	0.90	Fraud	**Fraud**

Feature engineering

Handling class imbalance – Algorithm-level approaches: robust imbalance models

- **Decision boundary optimization**
  - The decision boundary is influenced mainly by examples **close to the class separation**

Transaction	Risk Score	True Class	Prediction
T1	0.10	Normal	**Normal**
T2	0.15	Normal	**Normal**
T3	0.20	Normal	**Normal**
T4	0.30	Normal	**Normal**
T5	0.35	Normal	**Normal**
T6	0.40	Normal	**Normal**
T7	0.45	Normal	**Normal**
T8	0.50	Fraud	**Normal**
T9	0.70	Fraud	**Fraud**
T10	0.90	Fraud	**Fraud**

Boundary-defining samples:

- Majority examples far from the boundary (T1–T5) have little influence
- The model focuses on difficult cases near the separation

- Even though Normal transactions are the majority, the decision boundary is determined mainly by examples near the class separation
- Decision-boundary-based models can still work reasonably well under class imbalance

Feature engineering

Handling class imbalance – Hybrid approaches

- **Hybrid approaches** – combine both resampling and algorithm adjustments

  - **Resampling**: oversample minority, undersample majority, or both
  - **Algorithm adjustment**: class weights, cost-sensitive losses, or threshold tuning
  - Goal: preserve useful data while guiding the model to focus on minority samples
- Advantages:
  - Reduces bias toward the majority class
  - Useful for severe imbalance or high misclassification cost
- Considerations:
  - More complex than single-method approaches
  - Requires tuning of both resampling and algorithm parameters